Advanced Deployment of AAP 2.7

Version 1, 2026-07-31

Home

Introduction

Ansible Automation Platform (AAP) on OpenShift Container Platform (OCP) delivers enterprise-grade automation through the AAP Operator. A core feature of this deployment model is Automation Mesh, which enables scalable job distribution across remote execution nodes via peer-to-peer Receptor connections — all managed through the AAP UI without re-running the installer. In this training we will explore how to design and configure Automation Mesh in an operator-based environment, size your deployment using OpenShift Pod resource models, and troubleshoot common issues using OpenShift tooling.

Duration: 60 minutes

Objectives

On completing this course, you will:

- Understand how Automation Mesh works in an operator-based (OCP) deployment and Containerized deployment;
- Configure Instance Groups and remote execution nodes using the AAP UI and operator custom resources;
- Calculate sizing requirements expressed as Pod resource requests and limits;
- Troubleshoot AAP installation and mesh issues using the `oc` CLI and operator conditions and Containerized deployment.

Prerequisites

This course assumes that you have the following prior experience:

- Foundational knowledge of Linux, Containers, Kubernetes/OpenShift, and Ansible.
- Product Enablement: AAP 2.7 Fundamentals and Deployment.
- Familiarity with OpenShift concepts: namespaces, pods, service accounts, routes, and RBAC.

Contributors

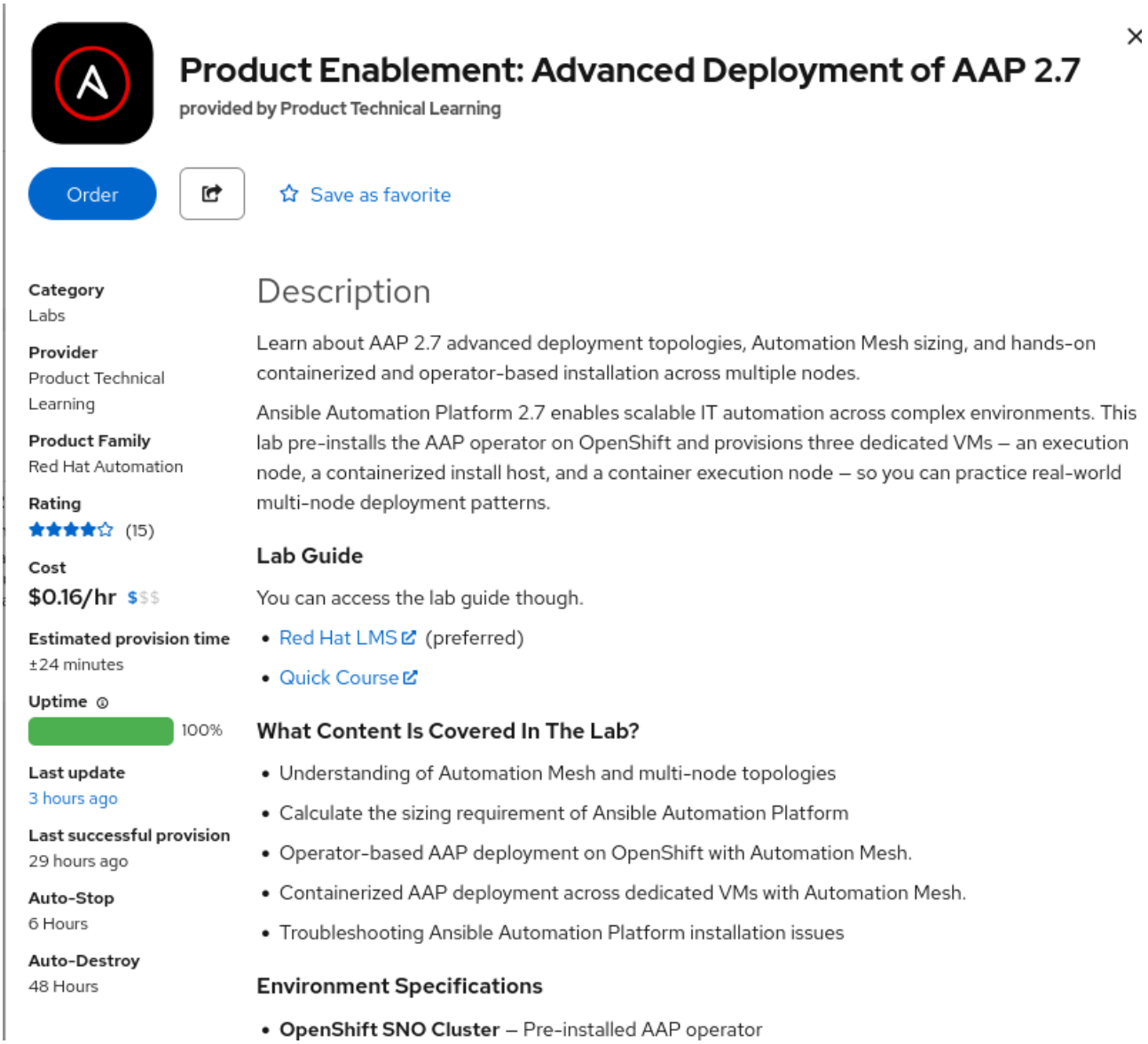
The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Amrinder Singh (Content Architect)
- Anna Swicegood (Learning Product Manager)
- Rupali Talwatkar (Coordinator)

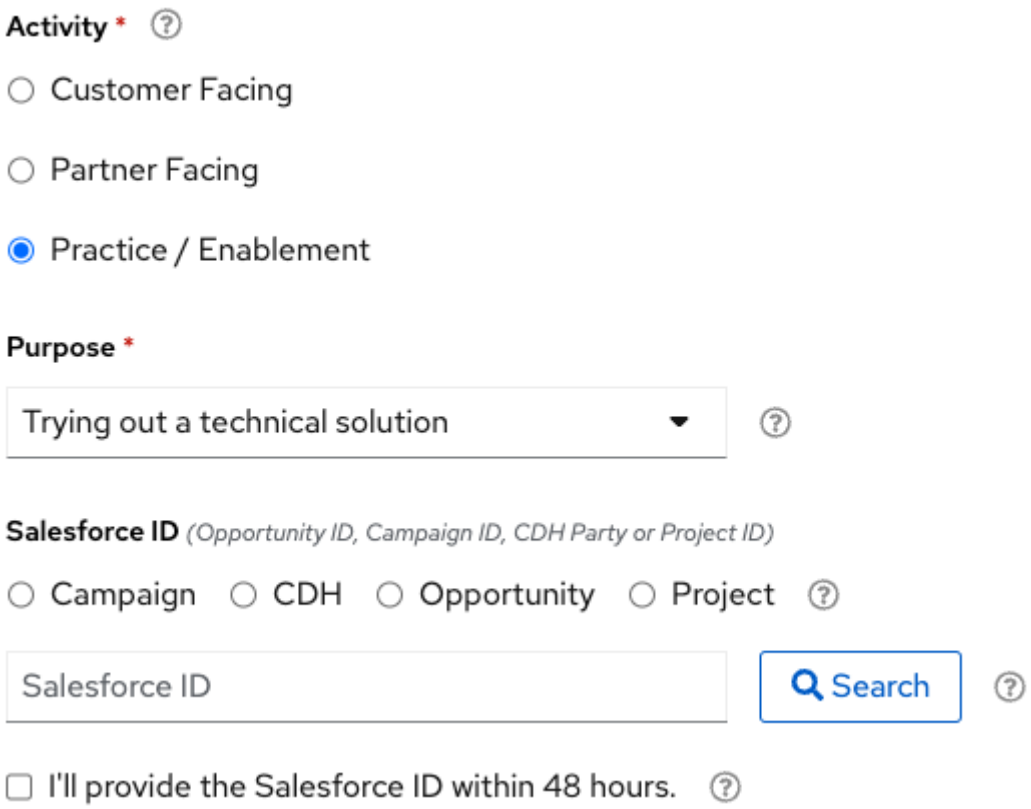
Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the RHDP portal.
2. **Search** for the catalog entry: **Advanced Deployment of Ansible Automation Platform 2.7**.



3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.



6. **Review the warning** at the bottom of the form and **check the box** labeled: **"I confirm that I understand the above warnings."**
7. **Click** the **Order** button to place your lab order.

Important Notes:

- This lab may take approximately **20 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

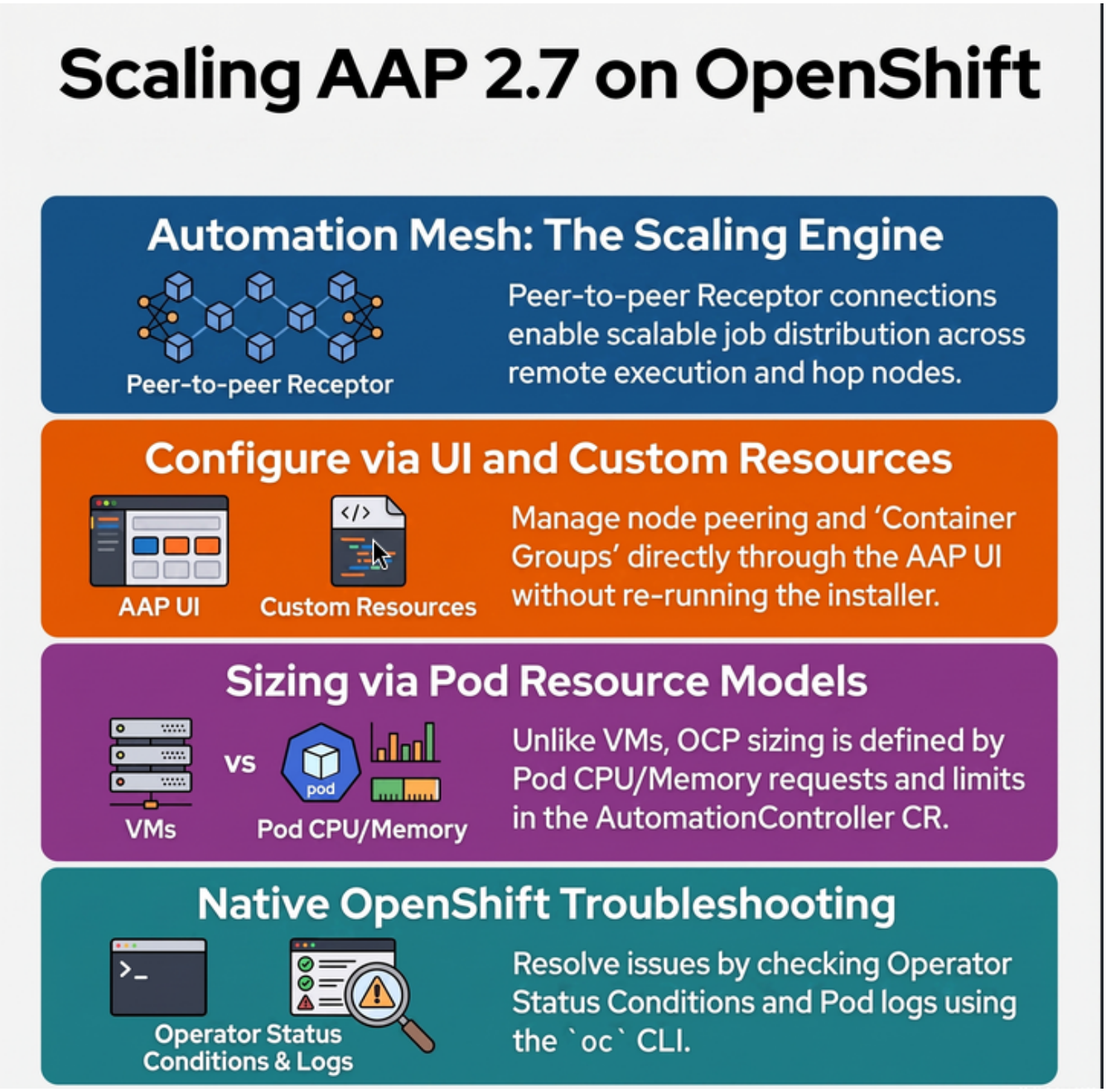
1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Advanced Deployment of Ansible Automation Platform 2.7 on OpenShift

Ansible Automation Platform on OpenShift Container Platform provides scalable, operator-managed IT automation. This course focuses on Automation Mesh in operator-based environments, where the Control Plane runs as pods inside the Kubernetes cluster and remote execution nodes peer to it via Receptor over TLS.



Participants will acquire practical skills in:

- Configuring Automation Mesh for an operator-based (OCP) deployment using Container Groups and the AAP UI.
- Adding remote execution and hop nodes using the `AutomationControllerMeshIngress` custom resource.

- Determining cluster sizing by translating fork and event calculations into Pod resource requests and limits.
- Troubleshooting common installation and mesh issues using `oc` CLI and operator status conditions.

Automation Mesh on OpenShift

Automation Mesh is an overlay network designed to enable the allocation of work among a large and dispersed group of workers known as Execution Nodes. This is achieved by creating peer-to-peer connections between nodes over existing networks. Receptor is the service that manages these connections.

Automation Mesh communications are encrypted with TLS, which allows securing traffic that moves across external networks such as the Internet, and enables AAP to achieve the following:

Key capabilities automation mesh introduces:

Capability	Description
Dynamic cluster capacity	Create, register, group, ungroup, and deregister nodes with minimal downtime
Plane separation	Scale playbook execution capacity independently from control plane capacity
Resilient deployment	Reconfigurable without outage; dynamically re-routes when outages occur
FIPS-compliant connectivity	Bi-directional, multi-hopped mesh communication
TLS encryption	All traffic traversing external networks is encrypted in transit

Automation mesh is useful for:

- Traversing difficult network topologies
- Bringing execution capabilities (the machine running `ansible-playbook`) closer to your target hosts

Control Plane on OpenShift

In an operator-based (OCP) deployment, the Control Plane is comprised of pods running inside the Kubernetes cluster — there are **no Hybrid or Control nodes** as in a VM-based deployment. The AAP Operator manages these pods and runs persistent Ansible Automation Platform services:

- Web server
- Task dispatcher
- Project updates
- Management jobs

The key difference from a VM-based deployment is that the Control Plane local to the Kubernetes cluster is made up of **Container Groups** — containers running on the cluster — rather than Hybrid or Control nodes.

Execution Plane on OpenShift

The Execution Plane consists of execution nodes that execute automation on behalf of the Control Plane and have no control functions. Hop nodes serve to route traffic. Execution Plane nodes only run user-space jobs, and may be geographically separated, with high latency, from the Control Plane.

- **Execution nodes**: Run jobs using `ansible-runner` with `podman` isolation. These are standalone RHEL VMs registered to the automation controller as type `execution` instances. They are only used to run jobs, not dispatch work or handle web requests.
- **Hop nodes**: Similar to a jump host, hop nodes route traffic to other execution nodes. Hop nodes cannot execute automation. They are registered in automation controller as an instance of type `hop`, handling inbound and outbound traffic for otherwise unreachable nodes in different or more strict networks.

In operator-based environments only `execution` and `hop` instance types can be created. Operator-based deployments do **not** support Control or Hybrid nodes.

Container Groups

Container Groups are a special type of Instance Group available in operator-based deployments. They enable automation controller to run jobs in containers provisioned on-demand as pods that exist only for the duration of the playbook run. This is known as the ephemeral execution model and ensures a clean environment for every job run.

Container Groups act as a pool of resources within an OpenShift namespace. You can create instance groups to point to an OpenShift container, using a service account and a bearer token to authenticate to the cluster.

Container groups do not use the capacity algorithm that normal execution nodes use. You set the number of forks at the job template level. The fork setting is passed directly to the container.

Peers

Peer relationships define the node-to-node connections that form the automation mesh. In an operator-based environment, peers are defined through the AAP UI for individual instances — not via an installer inventory file.

When you add an execution or hop node through the UI, you configure whether it peers directly from control nodes (**Peers from control nodes**) or is peered through a hop node.

Now let's move on to the practical implementation of Automation Mesh on OpenShift. In the next section, we will cover how to set up and configure the Control Plane and Execution Plane, as well as

how to establish peer relationships between nodes.

Automation Mesh Configuration on OpenShift hands-on Lab

In an operator-based AAP deployment, Automation Mesh is configured entirely through the AAP UI and OpenShift custom resources — not through installer inventory files. This section walks through the remote execution for extending the mesh beyond the cluster.

Lets setup the VM and connect it to Ansible Automation Platform for the execution node:

1. **OpenShift Console Access:**

   Open the OpenShift Console at `{openshift_console_url}` and log in with username `admin` and password `{common_admin_password}`.

   a. Navigate to **Virtualization** → **VirtualMachines**.

   b. Select namespace: `aap-lab`

   c. Click on `aap-lab-execution`

      [1.1] | *1.1.png*

   d. Click on the **Open Web Console** button.

      [1.2] | *1.2.png*

   e. Use the **Serial** tab for terminal access.

      [1.3] | *1.3.png*

2. Log in to the VM using username `lab-user` and password `{common_user_password}`. Copy and paste with CTRL+SHIFT+V in the terminal.
3. Register the system with Red Hat Subscription Management and attach the subscription to the system.

   ```
   sudo subscription-manager register
   ```

   ```
   username: <YOUR-RHN_LOGIN>
   password: <YOUR-RHN_PASSWORD>
   ```

```
aap-lab-execution login: lab-user
Password:
[lab-user@aap-lab-execution ~]$ sudo subscription-manager register
Registering to: subscription.rhsm.redhat.com:443/subscription
Username: rhn-support-amrsingh
Password:
The system has been registered with ID: e3990699-0e24-477d-9c63-e7c
The registered system name is: aap-lab-execution.headless.aap.svc.c
```

a. Enable the required repository for AAP 2.7:

```
sudo subscription-manager repos --enable ansible-automation-platform-2.7-for-rhel-10-x86_64-rpms
```

```
[lab-user@aap-lab-execution ~]$ sudo subscription-manager repos --enable ansible-automation-plat
Repository 'ansible-automation-platform-2.7-for-rhel-10-x86_64-rpms' is enabled for this system.
_
```

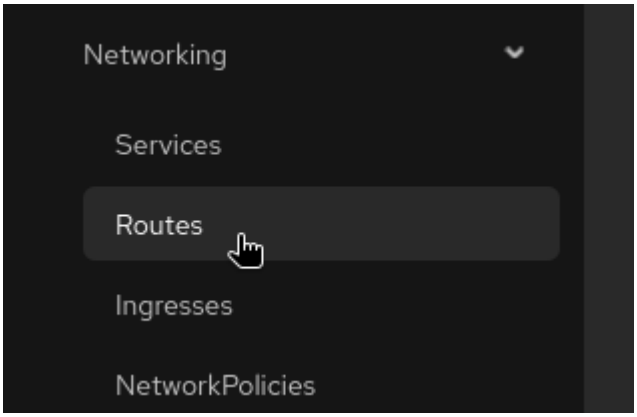
4. Install the required and useful utilities:

```
sudo dnf install -y vim ansible-core
```

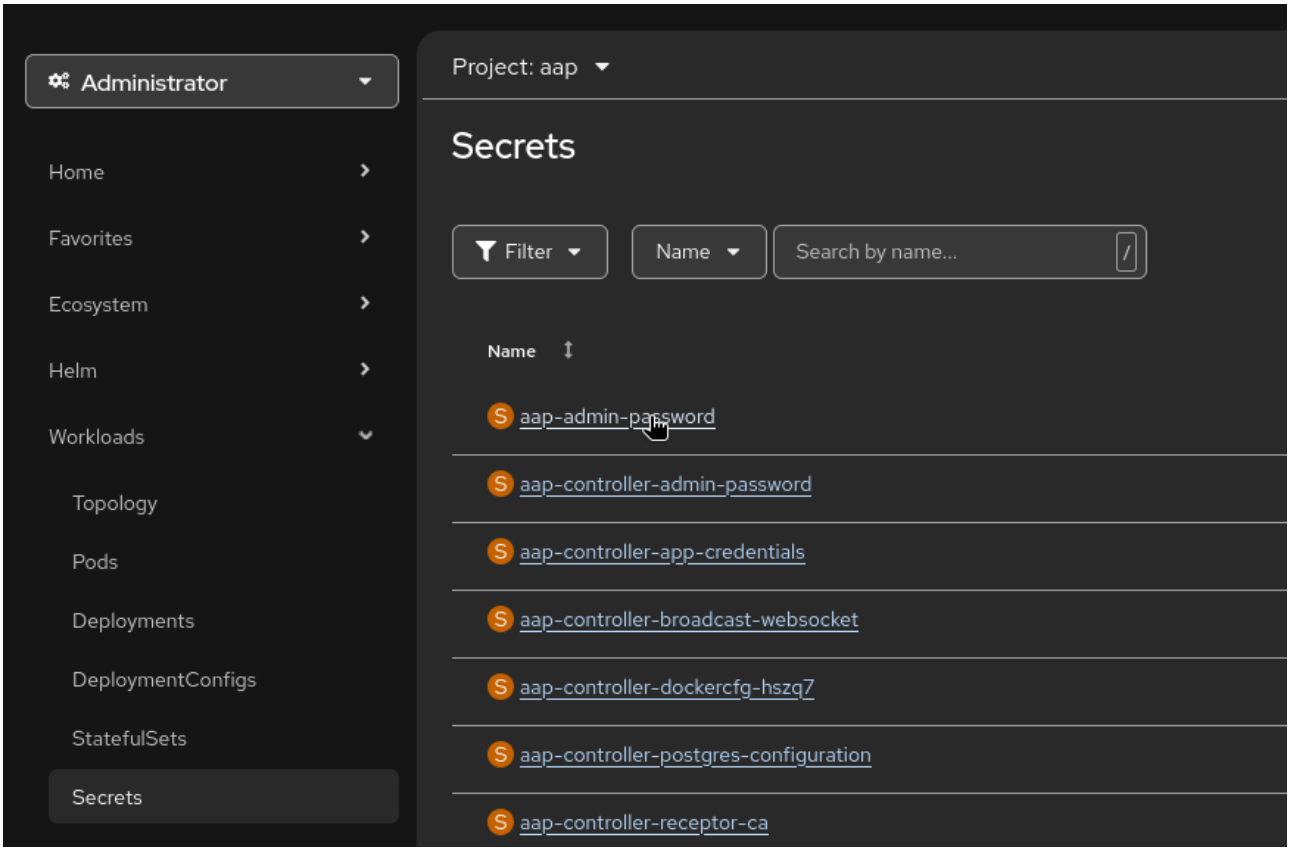
Install Receptor on the Execution Node:

Ansible Automation Platform UI Configuration:

1. Login to the AAP UI for that go to openshift cluster and then go to **Networking** > **Routes** > **Location for the aap Service** open the URL in the browser in the new tab.

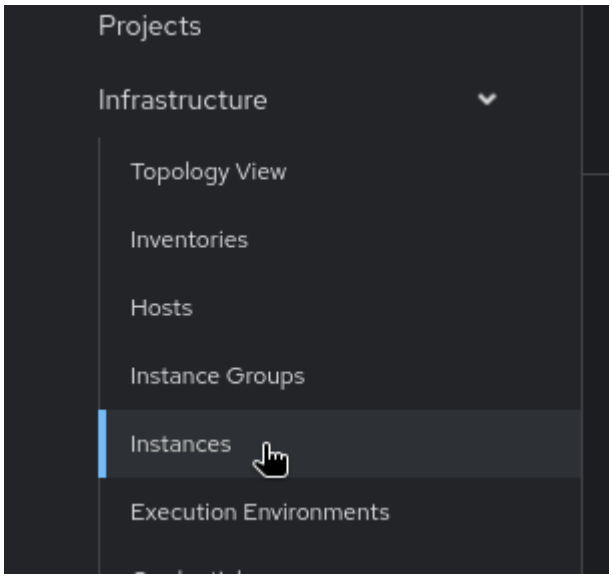


2. Go to **Workloads** > **Secrets** > **aap-admin-password** and copy the password to login to the AAP UI. for this lab you can use this for admin password: `{common_admin_password}`.



3. Download the Receptor install bundle from the AAP UI and copy it to the execution node. Use the following command to extract the bundle:

   a. From the navigation panel, select **Automation Execution** > **Infrastructure** > **Instances**.

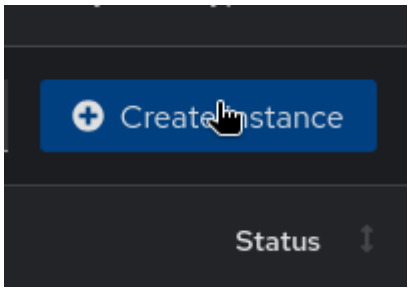


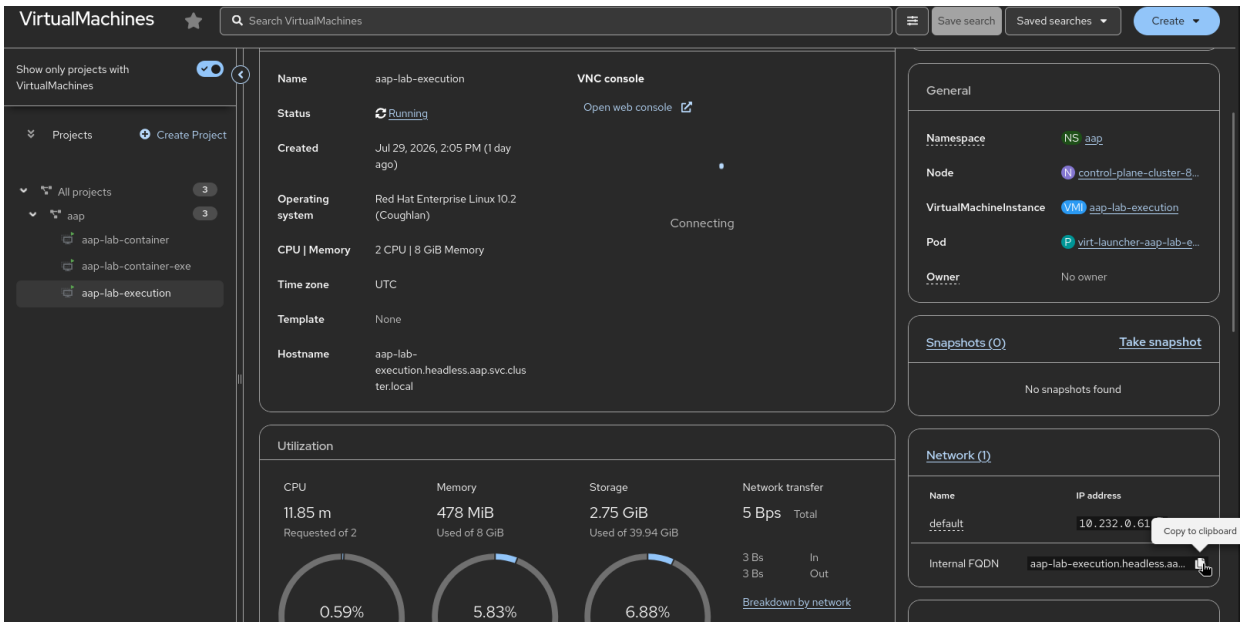
   b. On the **Instances** list page, click **Add instance**. Fill in the following fields:

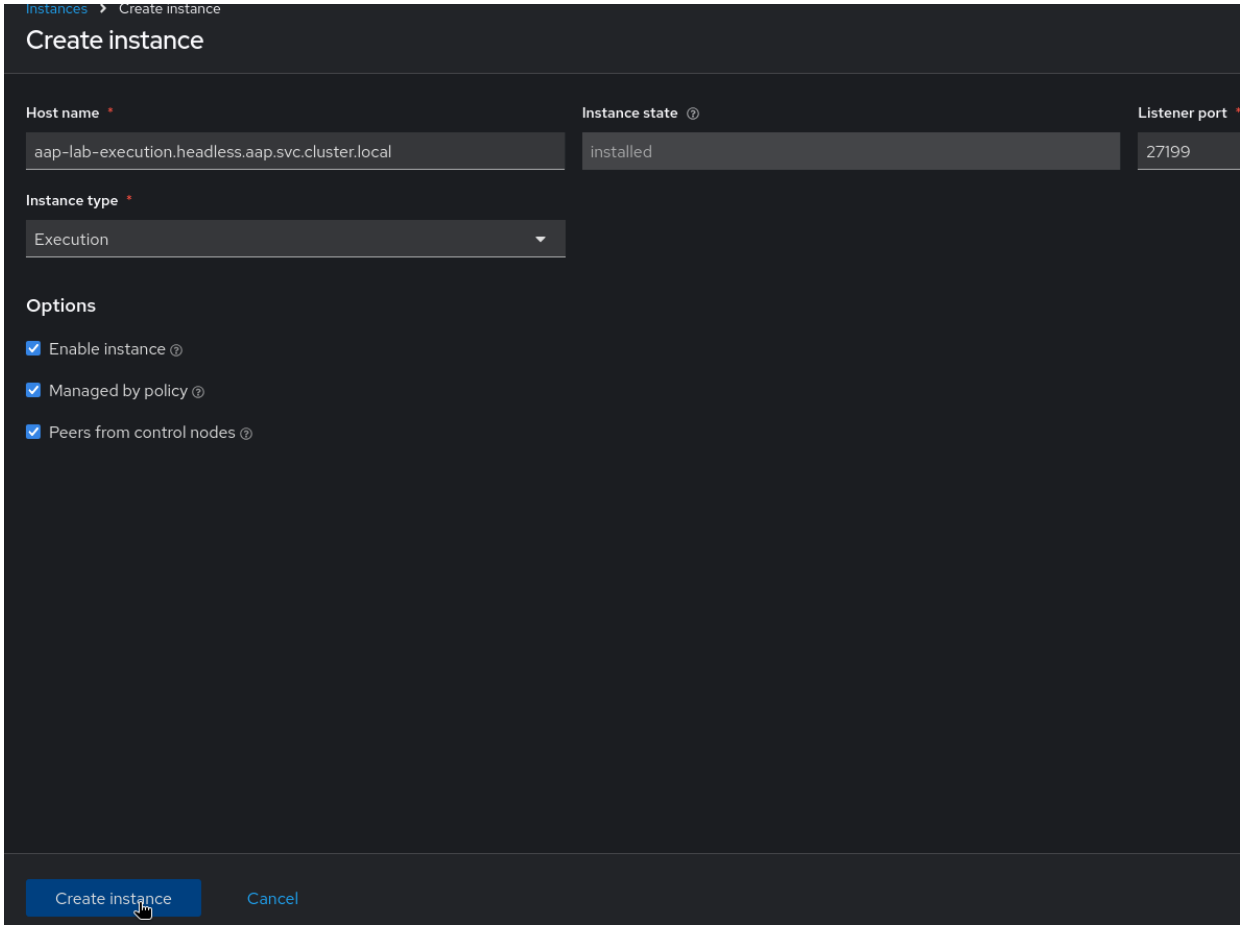
Field	Description
Host name	Fully qualified domain name (public DNS) or IP address. Use IP if private DNS cannot be resolved from the control cluster.
Description	(Optional) A description for the instance.

Instance state	Auto-populated (`Installed`); cannot be modified.
Listener port	Port for Receptor to listen on for incoming connections. Default: **27199**.
Instance type	Only `execution` and `hop` are supported on OpenShift operator deployments.

c. Add the VM `aap-lab-execution` FQDN from the Openshift cluster **Virtualization** > **VirtualMachines** > **aap-lab** > **aap-lab-execution** and the port `27199` to the instance and click **Save**.







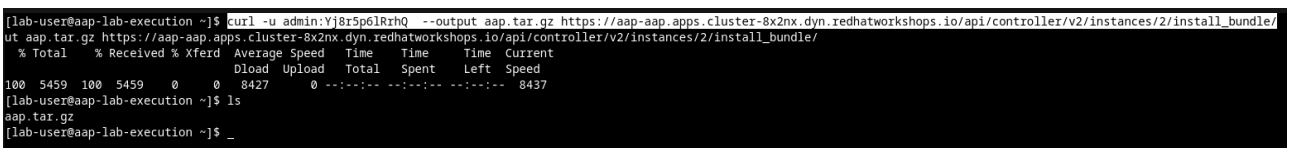
d. Copy the URL of the bundle from the AAP UI and download it on the execution node using the following command:

[1.12] | *1.12.png*

Setup Receptor on the Execution Node:

1. Download and Extract the bundle on the remote machine. The aap admin password is {common_admin_password}.

```
curl -u admin:<aap_admin_password> --output aap.tar.gz <Paste the URL of the bundle from the AAP UI>
```




```
tar -xzf aap.tar.gz
```

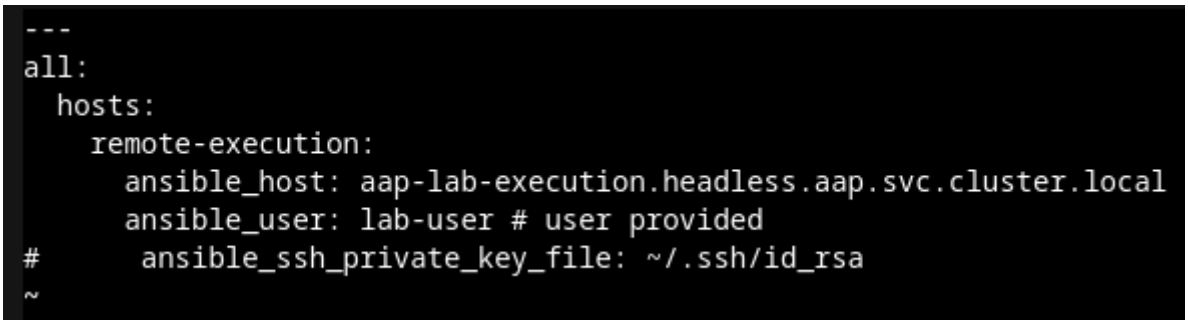
2. Move into the extracted directory and run the install script:

```
cd <extracted directory>
```

3. Edit `inventory.yml`

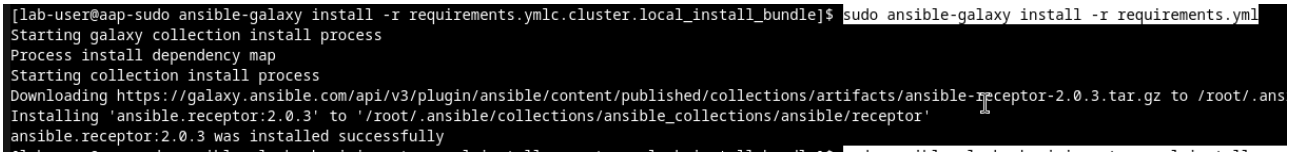
```
vim inventory.yml
```

```
all:
  hosts:
    remote-execution:
      ansible_host: <this-should-be-the-mesh-node-host-name>   # change to the mesh
node host name if its not
      ansible_user: lab-user # this should have the username of the remote
execution node
 #     ansible_ssh_private_key_file: ~/.ssh/<id_rsa>
```



4. Install the `ansible.receptor` collection:

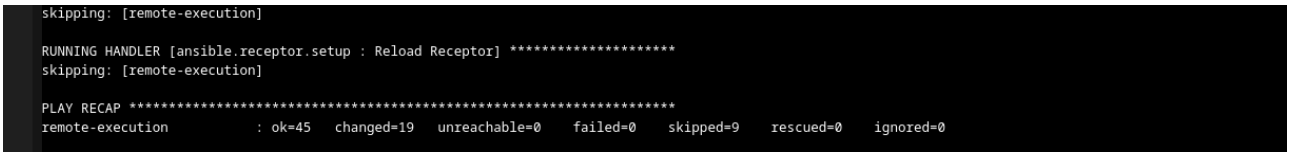
```
sudo ansible-galaxy install -r requirements.yml
```



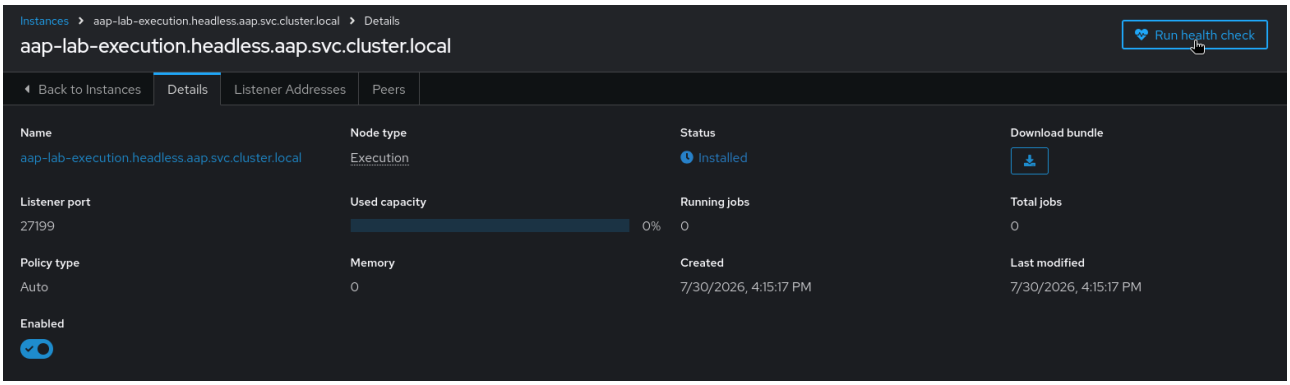

```
Open port 27199 for Receptor communication on the Firewall of the execution node
and the communication should be allowed bi-directionally between the execution node
and the control plane nodes.
```

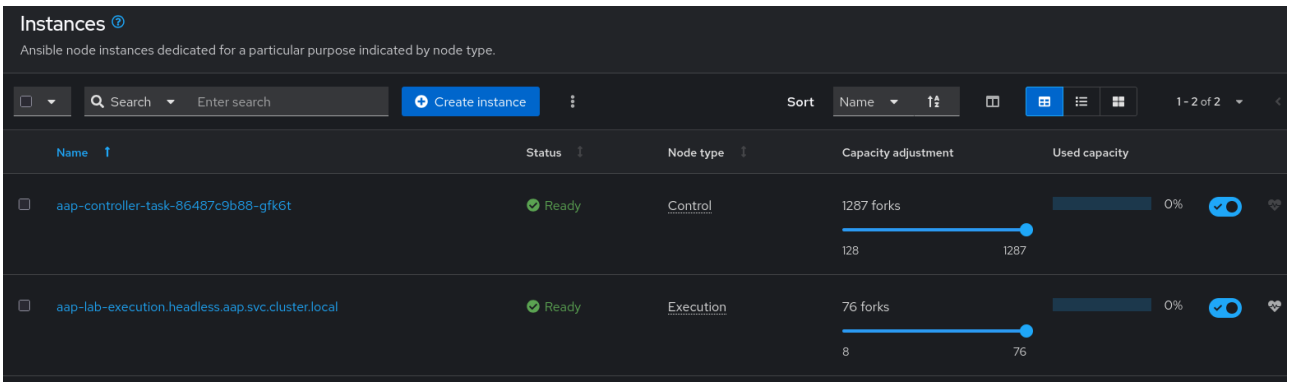
5. Run the installation playbook and provide the password: `{common_admin_password}` for the execution node when prompted. This will install Receptor on the execution node and configure it to connect to the control plane nodes.:

```
sudo ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory.yml
install_receptor.yml -k
```



6. Once the installation is complete successfully. Proceed to the next step to verify the connection between the execution node and the control plane nodes by clicking on the **Run Health Check** button in the AAP UI and soon the status will show **Ready** for the Execution node.



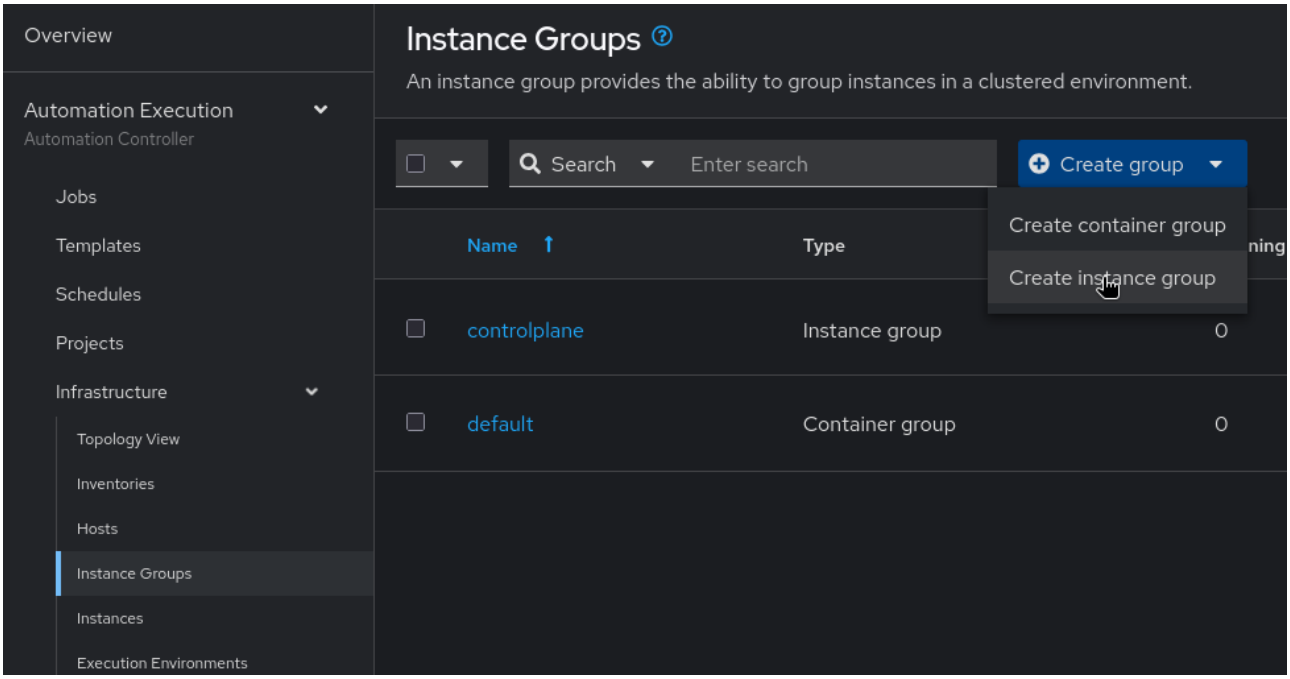


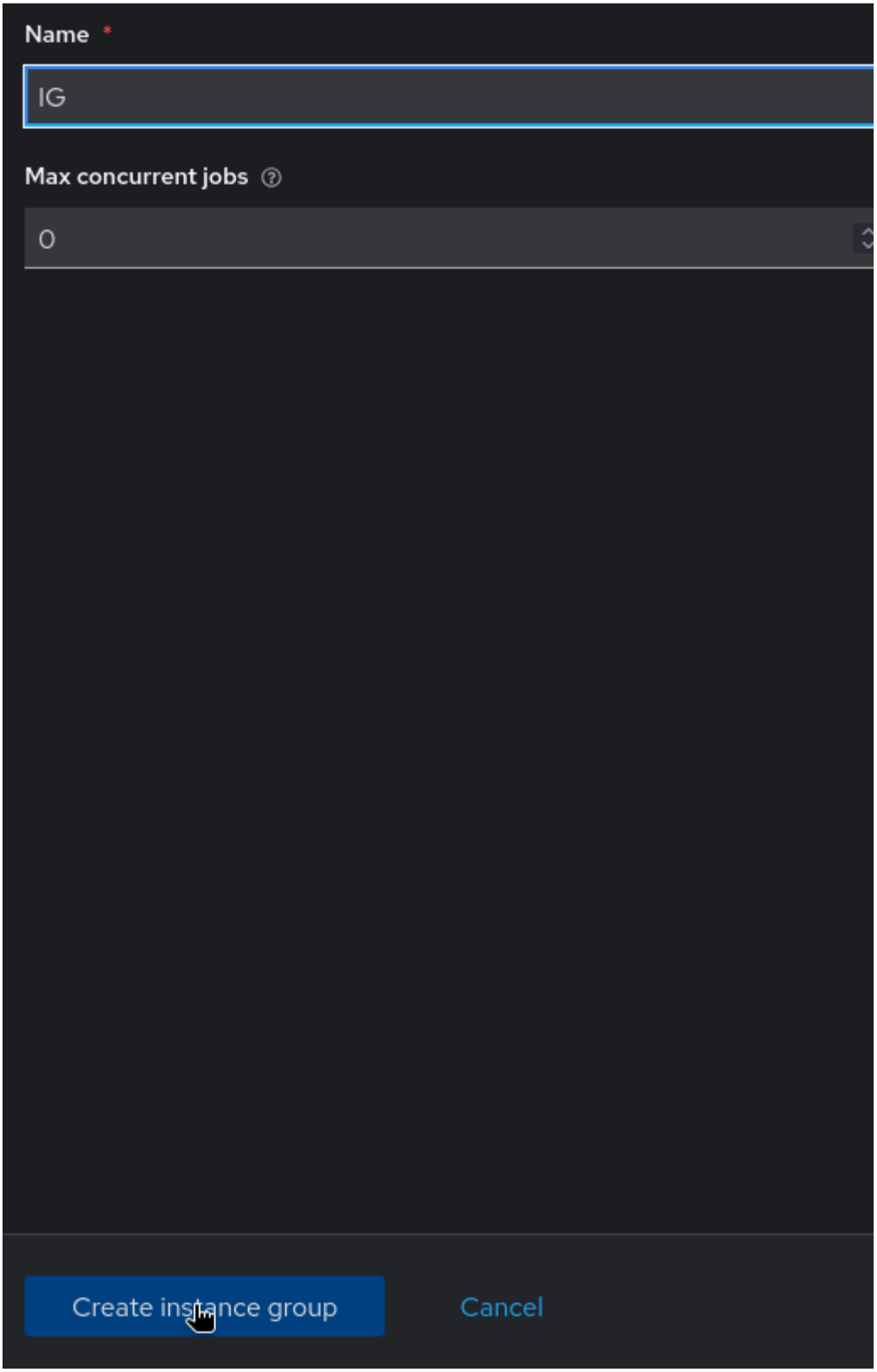
Let run a JOB on the execution node to verify if it will get executed. The job might be failing due to EE but the execution node should be the `aap-lab-execution` node that we have assigned.

Ansible Automation Platform 2.7 Job run on execution nodes:

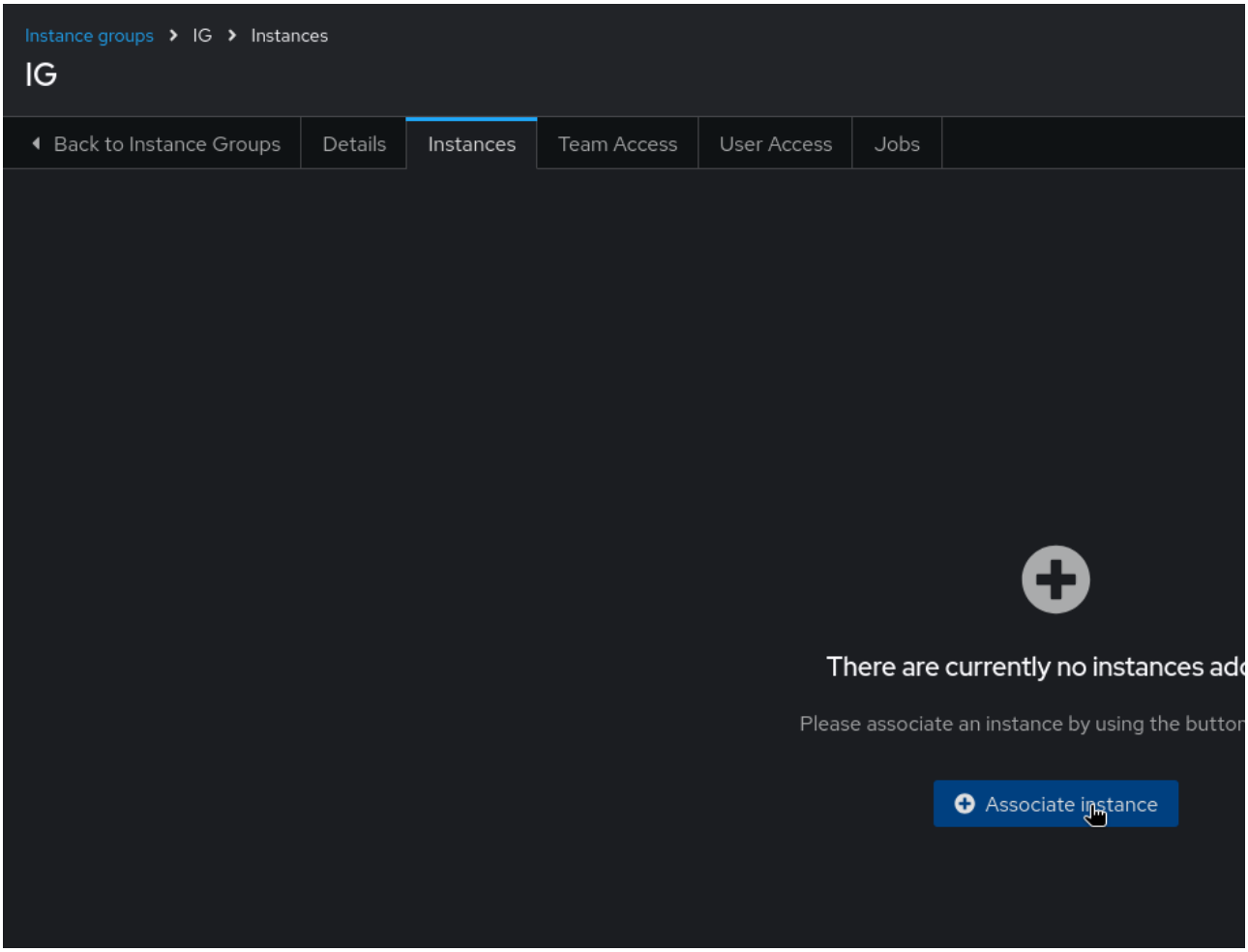
Part 1: Create a Instance Group:

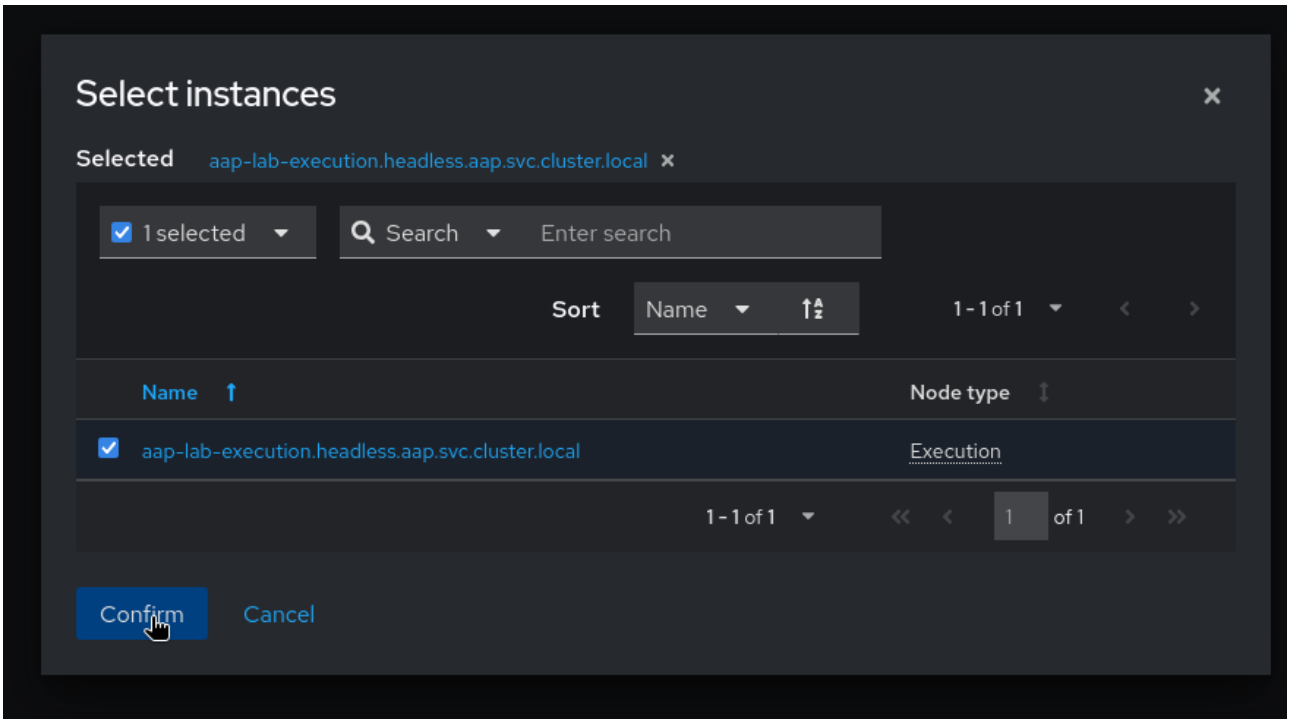
- Navigate to **Automation Execution** > **Infrastructure** > **Instance Groups**.
- Click **Create group** and select **Create instance Groups**.
- Enter `IG` as the name and then go to the **Instances** tab and add the Instance you created in the previous step. Click **Save**.


Overview
Automation Execution
Automation Controller
Jobs
Templates
Schedules
Projects
Infrastructure
Topology View
Inventories
Hosts
Instance Groups
Instances
Execution Environments
Instance Groups
An instance group provides the ability to group instances in a clustered environment.
Search
Enter search
Create group
Create container group
Create instance group
Name
Type
controlplane
Instance group
0
default
Container group
0



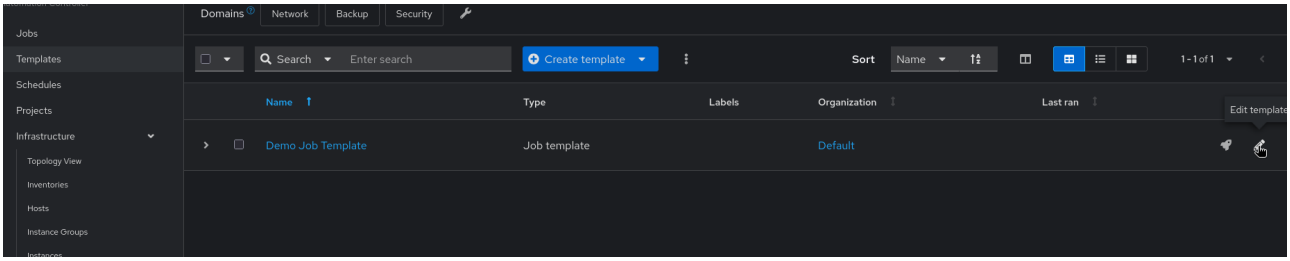
- Assign the Instance to this Instance Group and click on `aap-lab-execution.headless.aap.svc.cluster.local` and then Confirm the selection.



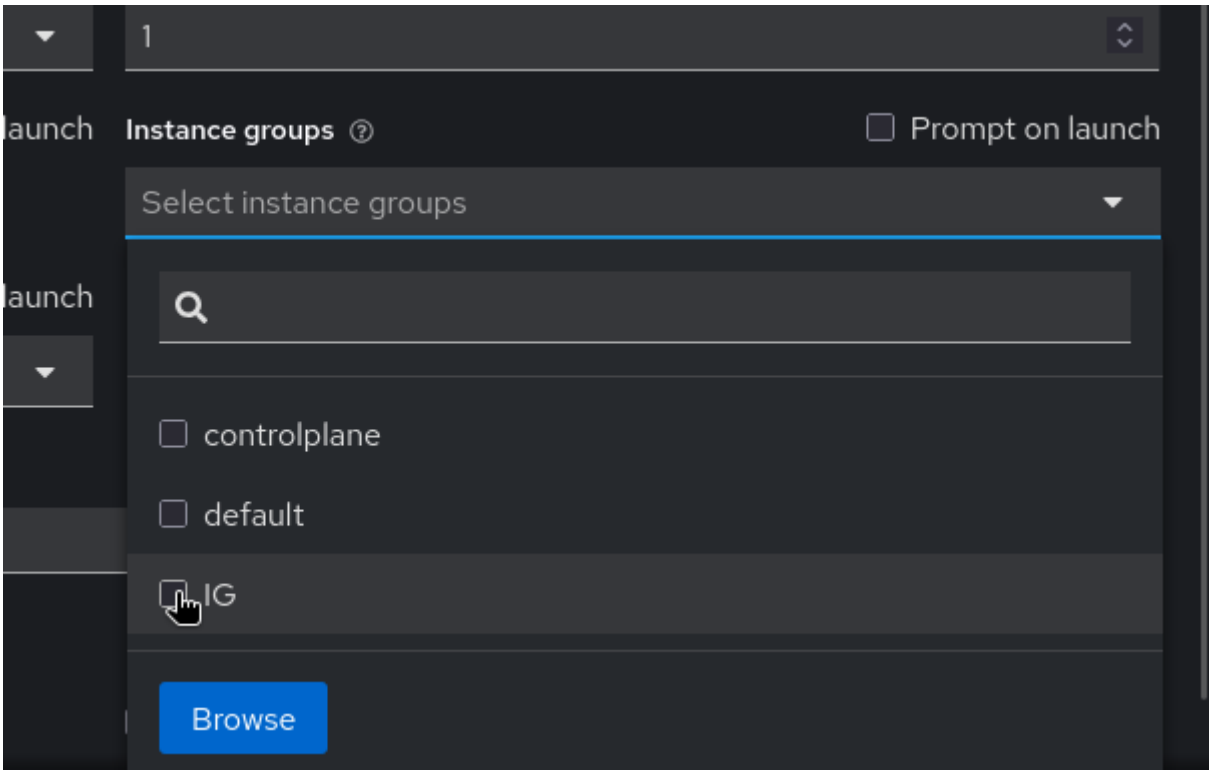


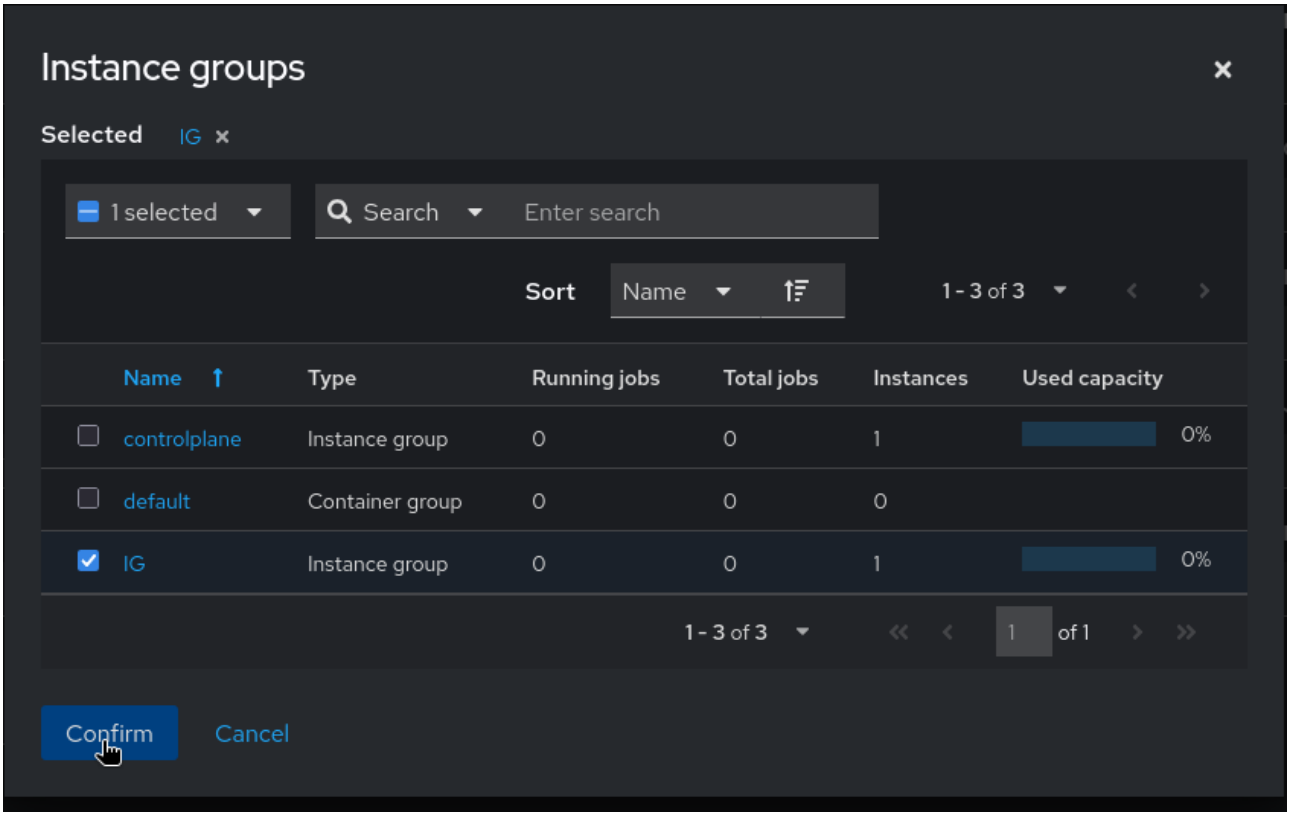
Part 2: Run a Job Using the Instance Group

1. In the AAP UI, navigate to **Automation Execution** > **Templates**.
2. Click the **edit** (pencil) icon on the Demo Job Template to open its settings.

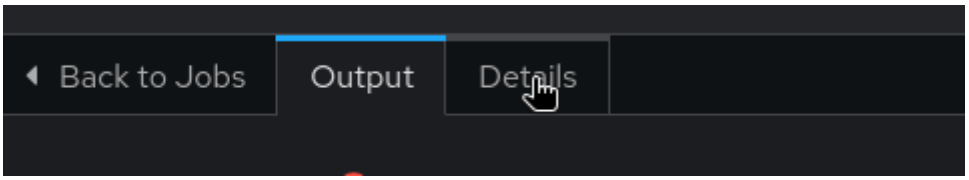


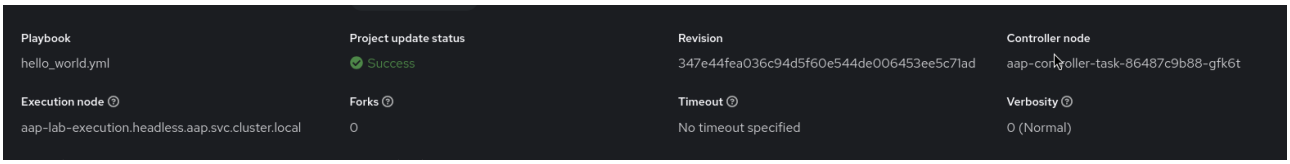
3. Click **Instance groups** and select `IG`, then save the template.





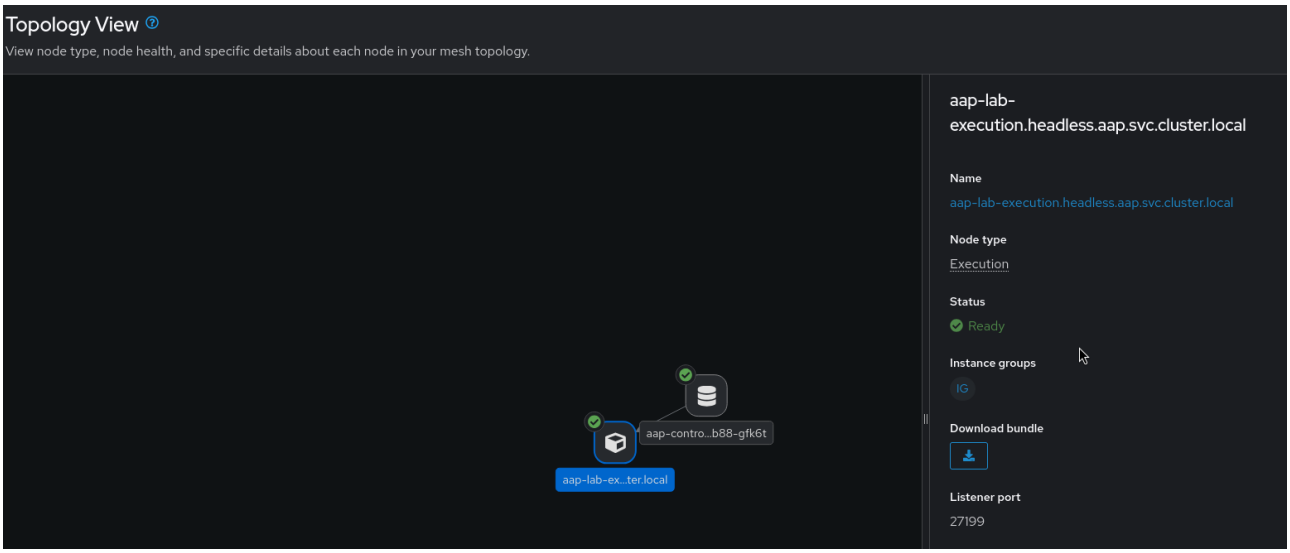
4. Launch the **Demo Job Template**.
5. In the AAP UI navigate to **Automation Execution** > **Jobs**, select the running job and check the **Details** section to verify it is executing via the Instance group via the execution node.





Part 3: View the Topology

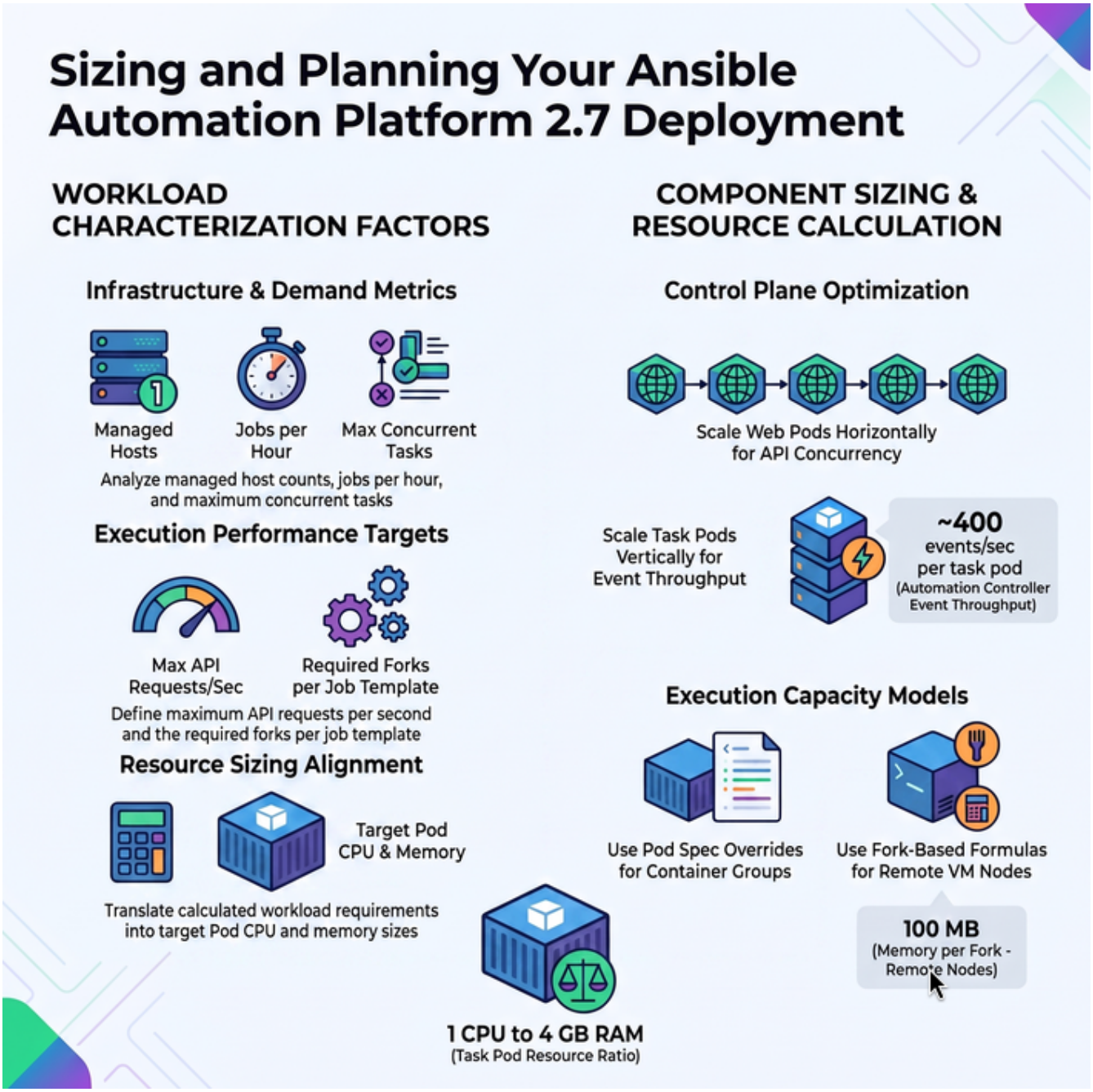
1. In the AAP UI, navigate to **Automation Execution** > **Infrastructure** > **Topology View**.
2. Verify the graphical representation shows your Instance Group connected to the Control Plane.
3. Refer to the **Legend** to identify node types and link statuses.



Calculate Sizing Requirements for an OpenShift Deployment

Having the right amount of resources for Ansible Automation Platform helps make the platform reliable and reduces cost. In an operator-based deployment on OpenShift, sizing is expressed as Pod resource **requests** and **limits** rather than VM specifications. The high-level points that should be considered during capacity planning are:

- Characterizing the workloads.
- Reviewing the capabilities of different component pods.
- Planning the deployment based on the requirements of the workload.



The same isolated factors apply as in a VM-based deployment:

- Number of managed hosts controlled by AAP.
- Jobs or tasks running per hour per host.
- Maximum number of concurrent jobs to support.
- Maximum number of forks set on jobs.
- Maximum API requests per second.
- Target Pod CPU and memory sizes.

Automation Controller Pods

The AAP Operator deploys automation controller as multiple pods. The two primary workload pods are:

- **web**: Handles the UI and REST API. Scales horizontally — add more web pod replicas to handle more concurrent API requests.
- **task**: Dispatches jobs, processes job events, and updates the database. CPU and memory on this pod determine control capacity (events per second).

Scale the `task` pod vertically (increase CPU and memory) to increase event processing throughput. Scale `web` pods horizontally to increase API concurrency.

Recommended ratio for the `task` pod: **1 CPU to 4 GB RAM**. Add CPU together with memory even if CPU is not a bottleneck — during high memory utilization the queue fills with unprocessed events, and higher CPU helps drain the queue faster.

Container Groups (Execution Capacity on OCP)

Container Groups run jobs as ephemeral pods. Execution capacity is determined by the resource limits set on each pod in the **Pod spec override**, not by a node's total memory. Key points:

- Forks are set at the job template level and passed directly to the container.
- Container groups do **not** use the fork capacity algorithm (memory / 100 MB) used by execution nodes.
- Use `LimitRange` objects in your namespace to set namespace-wide pod resource limits.
- Set `resources.requests` and `resources.limits` in the pod spec override for fine-grained control.

```
resources:
  requests:
    cpu: 250m
    memory: 100Mi
  limits:
    cpu: "2"
    memory: 2Gi
```

Remote Execution Nodes (RHEL VMs peered to OCP)

Remote execution nodes are RHEL VMs that extend the mesh beyond the OCP cluster. Their capacity is calculated the same way as in a VM-based deployment:

Memory Relative Capacity

`mem_capacity` is the maximum memory needed by one fork. The default value is 100 MB. 2 GB of memory is reserved for AAP services.

```
Example: 16 GB of available node memory → (16384 - 2048) / 100 ≈ 140 forks
```

CPU Relative Capacity

The baseline value is 1 CPU = 4 forks.

```
4 CPUs x 4 forks per CPU = 16 forks
```

The `capacity_adjustment` value (0 to 1) determines whether fork count is closer to the CPU-bound or memory-bound limit. At 0.5:

```
Capacity = [140 forks memory x 0.50] + [16 forks CPU x 0.50]
         = 70 + 8 = 78 forks
```

Automation Hub (Private Automation Hub)

Private Automation Hub provides Execution Environments and collections, so it has modest CPU and memory needs. Storage sizing dominates:

- Number of Execution Environments stored.
- Size of each Execution Environment.
- Number of versions of each Execution Environment.

In an OCP deployment, storage is provided by Persistent Volume Claims (PVCs). Ensure the selected StorageClass supports `ReadWriteMany` if multiple hub pods are needed, or `ReadWriteOnce` for single-pod deployments.

Event-Driven Ansible Controller (AutomationEDA)

Event-Driven Ansible workloads are sized based on:

- Number of simultaneous rulebook activations.
- Number of events received per minute.

In an OCP deployment, rulebook activation limits are configured via the `AutomationEDA` custom resource spec rather than a file-based settings file. The default is 12 rulebook activations per node and 24 activations total. Memory per activation container is set as a Pod resource limit in the CR — the equivalent of the `PODMAN_MEM_LIMIT` setting in non-OCP deployments.

Database (PostgreSQL)

The AAP Operator integrates with managed PostgreSQL (via Crunchy Data operator or an external database). Database Pod sizing should match or exceed the higher of the Control Plane or Execution Plane resource requirements:

- Database storage: see sizing example for formulas.
- Database RAM and CPU: align with the `task` pod sizing.

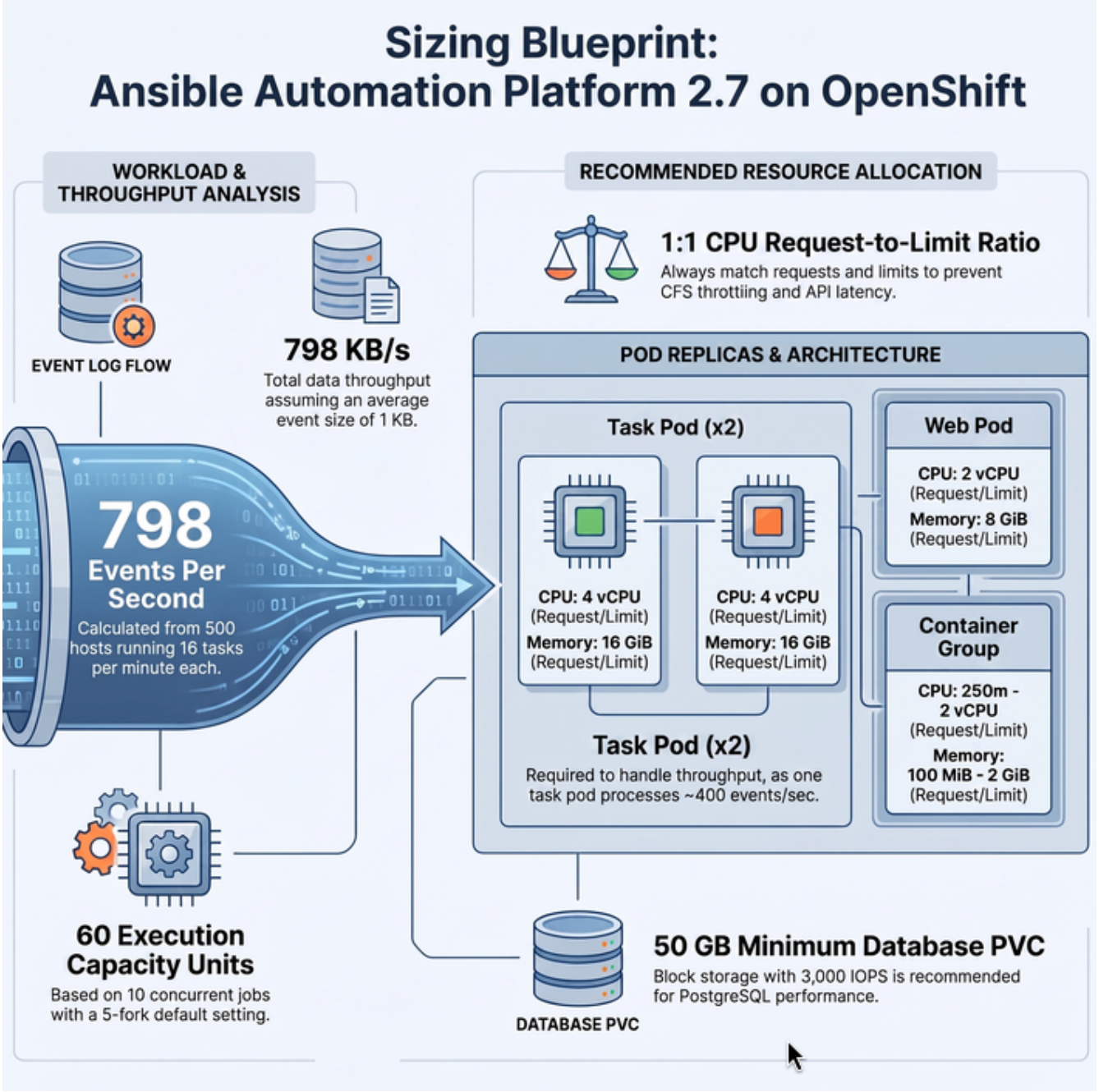
Example: Calculating Sizing for an OpenShift Deployment

At this point we should have a good idea how to calculate capacity and what factors to consider. Let's go through the same planning exercise applied to an OCP deployment.

For this example, the cluster must support the following capacity:

- 500 managed hosts
- 1,000 tasks per hour per host (16 tasks per minute per host)
- 10 concurrent jobs
- Forks set to 5 on playbooks (the default)
- The average event size is 1 KB

Target pod sizes: 4 vCPU and 16 GB RAM each.



Execution Capacity Calculation

Execution Capacity = (Number of jobs × forks value) + (Number of jobs × 1 base task impact)

```
(10 jobs x 5 forks) + (10 jobs x 1) = 60 execution capacity units
```

For a Container Group, this means each pod spec should allow at least 5 forks. Set the fork limit at the job template level. No special node sizing formula applies — forks are passed directly to the pod.

For a remote execution node peered to OCP:

```
16 GB RAM → (16384 - 2048) / 100 = 140 forks available
140 forks > 60 required → 1 execution node is sufficient
```

Two nodes are recommended for redundancy.

Control Capacity Calculation

```
Total tasks/min = 500 managed hosts x 16 tasks/min = 8,000 tasks/min = 133 tasks/sec
```

Assuming 6 events per task:

```
133 tasks/sec x 6 events/task = 798 events/sec
798 events x 1 KB/event = 798 KB/sec of event log data
```

The automation controller `task` pod must be able to process approximately 798 events per second. The default automation controller throughput is ~400 events/second per `task` pod replica.

Therefore, for this workload **2 task pod replicas** are recommended. Set this in the `AutomationController` CR:

```
apiVersion: automationcontroller.ansible.com/v1beta1
kind: AutomationController
metadata:
  name: aap-controller
  namespace: aap
spec:
  replicas: 2
  task_resource_requirements:
    requests:
      cpu: "4"
      memory: 16Gi
    limits:
      cpu: "4"
      memory: 16Gi
  web_resource_requirements:
    requests:
      cpu: "2"
      memory: 8Gi
    limits:
      cpu: "2"
      memory: 8Gi
```

In the `AutomationController` CR, `replicas` controls the number of automation controller instances (task pods). The `web` pod scales independently and handles API and UI requests. Vertical scaling (more CPU/RAM) on the task pod is the primary lever for event throughput. Always set CPU `requests` equal to CPU `limits` to prevent CFS throttling — as shown in the YAML above.

Database Sizing

```
Database size for facts  = Facts size per host x number of hosts / 1024
                         = 50 KB x 500 / 1024 ≈ 24 MB

Database size for jobs   = hosts x jobs/host/day x retention days x events x event
size / 1024
                         = 500 x 24 x 30 x 6 x 1 KB / 1024 ≈ 2,109 MB ≈ 2 GB (30-day
retention)
```

For an OCP deployment, provision a PVC for PostgreSQL sized to at least **50 GB** to account for growth and WAL logs.

Summary

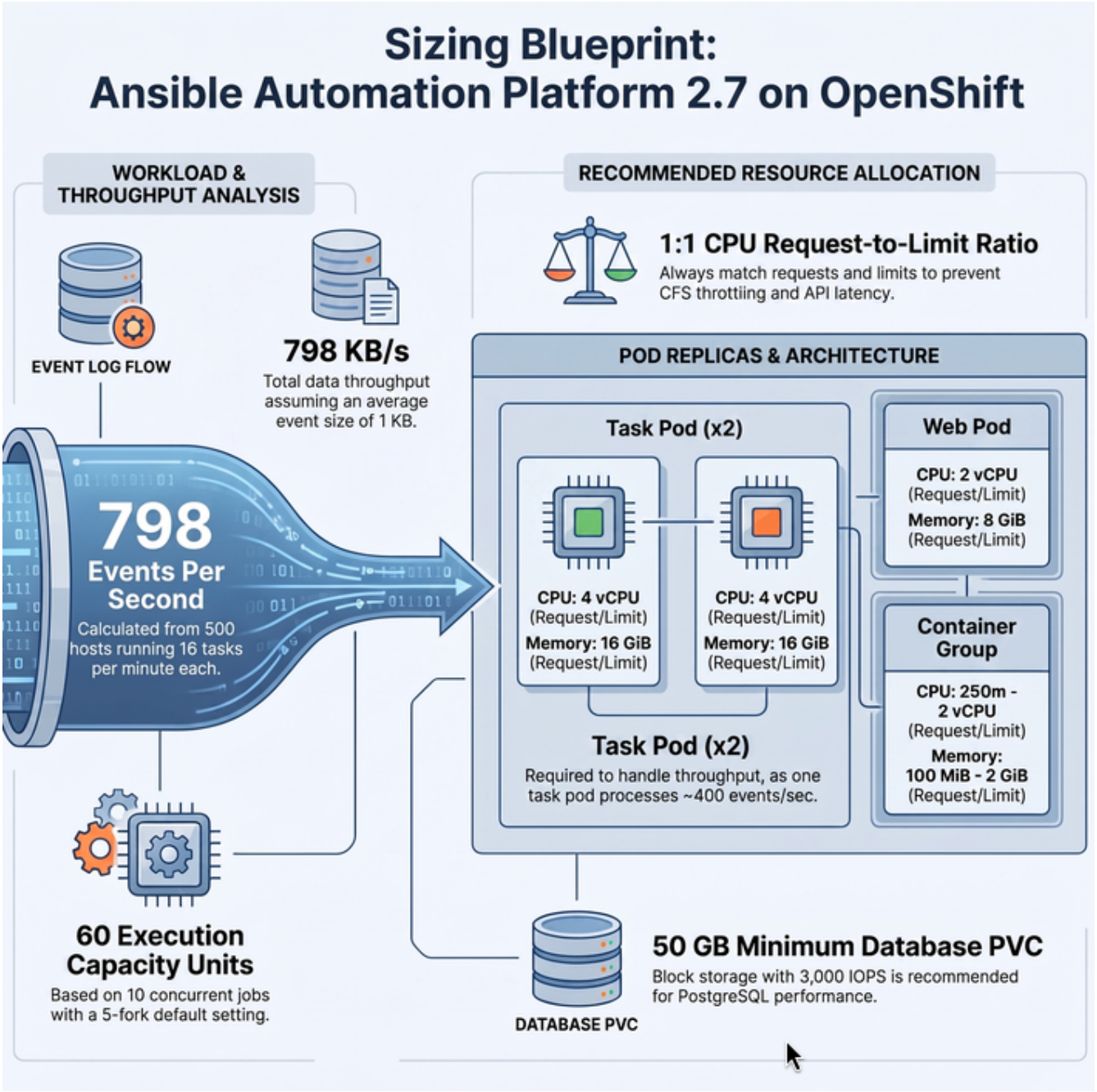
To summarize, for this workload on OpenShift:

- **Automation Controller**: 2 replicas, 4 vCPU / 16 GB RAM per task pod; 2 vCPU / 8 GB RAM per web pod.
- **Container Group (in-cluster execution)**: Pod spec with resources appropriate to the fork count (250m–2 CPU, 100Mi–2Gi per worker pod).
- **Remote Execution Node (if needed)**: 1–2 RHEL VMs, 4 vCPU / 16 GB RAM each.
- **PostgreSQL PVC**: 50 GB minimum.

For Automation Hub and EDA, start with the default operator sizing and scale based on observed storage and activation usage.

Performance Considerations for OpenShift Deployments

In the previous section we discussed how to calculate node size. The following default parameters and OCP-specific factors help with planning control and execution capacity.



The Tuning of the parameters needs careful consideration and should be done only after measuring the actual workload and performance of the system. The default values are based on typical workloads and may not be optimal for all environments.

Default Parameters

The following parameters are unlikely to need changes unless you have a specific reason and measured value:

Parameter	Default value
Facts size per host	50 KB
Size per event	1 KB
Events per task	6–10

Parameter	Default value
Memory per fork (execution nodes)	100 MB
Forks per CPU (execution nodes)	4
Capacity Adjustment	1.0 (select highest value)
Execution Environment average size	450 MB
Automation Controller event throughput (per task pod)	~400 events/sec
Automation Controller RAM per event fork	10 KB
Automation Controller CPU per event fork	0.0001
Concurrent API calls per controller web pod	100 (up to 300 allowed)
Controller/execution base job capacity per job run	1

In an OCP deployment, "Memory per fork" applies to remote execution nodes only. Container Group pods do not use the fork capacity formula — set fork limits directly in the pod spec or at the job template level.

Factors That Vary Across Environments

- Number of Ansible managed hosts/nodes.
- Number of jobs per host per day.
- Tasks run per playbook or job including imported roles/tasks/playbooks.
- Whether jobs run for a specific time period or 24×7.

CPU Throttling Prevention

In OpenShift Container Platform, setting a CPU `limit` without an equal CPU `request` causes the kernel CFS scheduler to throttle the container even when the node has spare CPU capacity. This throttling directly increases API latency and slows job event processing.

Rule: always set CPU `requests` equal to CPU `limits` for all AAP pods.

```
task_resource_requirements:
  requests:
    cpu: "4"
    memory: 16Gi
  limits:
    cpu: "4"        # must equal requests.cpu to avoid CFS throttling
    memory: 16Gi
```

Monitor CPU throttling with the Prometheus metric:

```
container_cpu_cfs_throttled_seconds_total
```

If this metric is non-zero and rising, increase the CPU `request` and the matching `limit` on the affected pod. Do not raise only the limit — that is the source of the throttling.

Manage Live Event Streams to the UI

The automation controller UI receives real-time job updates over WebSocket connections. Under heavy concurrent workloads, live event streaming increases load on the controller service.

Disable live streaming

Set `UI_LIVE_UPDATES_ENABLED` to `false` in the automation controller configuration. When disabled, the job detail page no longer auto-updates as events arrive — users must refresh manually to see current status.

```
extra_settings:
  - setting: UI_LIVE_UPDATES_ENABLED
    value: "false"
```

Tune event rate and size

The following settings control the volume of event data sent to the UI. Reducing these values lowers controller load at the cost of less granular live output:

Setting	Default	Effect
`EVENT_STDOUT_MAX_BYTES_DISPLAY`	1024	Maximum bytes of stdout displayed per event in the UI
`MAX_WEBSOCKET_EVENT_RATE`	30	Maximum events per second sent over WebSocket per job
`JOB_EVENT_BUFFER_SECONDS`	1	Time events are buffered before being sent to the UI

Job Type Impact on Capacity

Not all jobs consume the same controller capacity. Plan your sizing based on the actual workload mix:

Job type	Capacity impact	Notes
Standard jobs	1 fork = 1 capacity unit	The baseline — most playbook runs
Workflow jobs	Low on controller	Orchestrate child jobs; controller overhead is minimal but each child job consumes its own capacity

Job type	Capacity impact	Notes
Sliced jobs	1 unit per slice	A sliced job with 5 slices consumes 5 capacity units simultaneously
Bulk jobs	Sum of all child jobs	High total capacity; schedule during low-traffic windows
System jobs	Variable	Fact cleanup, project sync, inventory sync — compete with user jobs for task pod capacity

Project synchronizations and inventory syncs run as system jobs. During heavy project sync periods, reduce the number of concurrent jobs or schedule syncs during off-peak hours to avoid starving user workloads.

OpenShift-Specific Performance Considerations

Horizontal Pod Autoscaling

The `web` pod handles all UI and REST API traffic and scales horizontally — increase the `replicas` field in the `AutomationController` CR to add more web pod replicas. The `task` pod processes job events and scales vertically — increase its CPU and memory requests and limits to raise event throughput. When scaling `replicas`, also raise `web_resource_requirements` if API latency is the bottleneck.

Resource Quotas and LimitRanges

Ensure the AAP namespace does not have a `ResourceQuota` or `LimitRange` that caps pod memory below your sizing requirements. Quota exhaustion causes pod scheduling failures that appear as `Pending` pod status.

```
oc describe quota -n aap
oc describe limitrange -n aap
```

StorageClass and IOPS

Database and project storage performance on OCP depends on the underlying `StorageClass`. For production deployments, use a StorageClass backed by block storage. Provision at least 3,000 IOPS for the PostgreSQL PVC. Avoid `ReadWriteOnce` PVCs if you plan to run multiple hub or controller replicas — those require `ReadWriteMany`.

WebSocket Load Balancing

When running multiple `web` pod replicas, WebSocket connections require sticky sessions — the same client must be routed to the same pod for the life of the connection. Configure source-based load balancing on the OpenShift Route:

```
oc annotate route aap-controller \
  haproxy.router.openshift.io/balance=source
```

The `source` algorithm pins a client to the same backend pod based on source IP, maintaining WebSocket session continuity across reconnects.

ImagePullSecrets for Secured Registries

If your Execution Environments are stored in a private registry other than the default `registry.redhat.io`, configure an `imagePullSecret` in the `AutomationController` CR or in the pod spec override of the Container Group:

```
oc create secret docker-registry my-registry-secret \
  --docker-server=<registry-host> \
  --docker-username=<username> \
  --docker-password=<password> \
  -n aap
```

```
spec:
  image_pull_secrets:
    - name: my-registry-secret
```

Container Group Pod Overhead

Each Container Group job pod incurs Kubernetes scheduling overhead (approximately 2–5 seconds for pod start). For very short-lived playbooks, a persistent remote execution node provides better throughput than Container Groups, because the receptor connection is always up and there is no scheduling delay per job.

Key Performance Indicators

Monitor these metrics to determine when scaling or tuning is needed:

Indicator	Signal	Action
High API latency	Response time > 2 seconds for UI or API calls	Scale web pod replicas; check CPU throttling; check database connection pool
High CPU utilization	Task pod CPU sustained > 80%	Increase task pod CPU requests and limits equally to avoid throttling
502 Bad Gateway	Returned by platform gateway	Scale gateway replicas; check backend pod health
503 Service Unavailable	Returned by automation controller	Scale web pod replicas; check if task pod is overloaded
429 Too Many Requests	Returned by platform gateway	Rate-limiting triggered; use token-based API access; reduce polling frequency in clients

Indicator	Signal	Action
`container_cpu_cfs_throttled_seconds_total` rising	Prometheus metric	Set CPU `requests` equal to CPU `limits` on the affected pod

OpenShift Route Timeout

Long-running API requests — such as job launch polling or large inventory syncs — can exceed the default HAProxy Route timeout of 30 seconds, causing the client to receive a 504 error even though the backend is still working. Increase the timeout with the following annotation on each AAP Route:

```
oc annotate route aap-controller \
  haproxy.router.openshift.io/timeout=300s
```

Apply to each route that clients access (controller, hub, EDA gateway). Clients should also configure a matching or longer timeout on their side to avoid premature disconnection.

Helpful Formulas

```
Total Jobs = (Number of Hosts x Jobs per Host per Day) ÷ Hosts Required per Job

Job Duration = Job Duration per Host x CEILING(Hosts per Job ÷ Parallel Forks)

Workload Utilization = (Jobs per Day x Job Duration) ÷ Automation Hours per Day

Polling Load = (Jobs per Day x Job Duration) ÷ Polling Interval

API Utilization = (API Calls per Day x API Call Duration) ÷ Automation Hours per Day

Database size for facts  = Facts size per host x number of hosts / 1024

Database size for jobs = number of hosts x jobs per host per day x number of days
                         to keep the data x number of events x event size / 1024
```

Database RAM and CPU should be sized similarly to the `task` pod or Execution Plane node, whichever is higher.

Troubleshooting AAP Installation Issues on OpenShift

In an operator-based deployment, the AAP Operator reconciles custom resources and manages pod lifecycle. Most installation and configuration issues surface as failed operator reconciliation conditions, pods stuck in `Pending` or `CrashLoopBackOff`, or custom resources that never reach a `Running` state. The basic rule of thumb: check the operator conditions first, then the pod logs.

- Login to the `{bastion_public_hostname}` using the ssh command of the **Linux Developer Host** host from the RHDP login page and perform the following steps to know how to troubleshoot

the AAP installation issues on OpenShift.

- Login to the OpenShift cluster using the command:

```
oc login -u admin -p admin {openshift_api_url}
```

Step 1: Check the Operator CSV Status

Verify the AAP Operator is installed and healthy:

```
oc get csv -n aap
```

The `PHASE` column should show `Succeeded`. If it shows `Failed` or `Pending`, describe the CSV for details:

```
oc describe csv <csv-name> -n aap
```

Step 2: Check Custom Resource Conditions

Check the status of your `AutomationController` CR:

```
oc describe automationcontroller aap-controller -n aap
```

Look at the `Status.Conditions` section. Common conditions include:

- `Running`: Controller is up and healthy.
- `Failure`: The operator encountered an error — the `message` field describes the cause.
- `Degraded`: Some components are not ready.

Similarly for other CRs:

```
oc describe automationhub aap-hub -n aap
oc describe automationeda aap-eda -n aap
```

Step 3: Check Pod Status and Logs

List all pods in the AAP namespace:

```
oc get pods -n aap
```

For pods in `Pending` state, check events:

```
oc describe pod <pod-name> -n aap
```

For pods in `CrashLoopBackOff` or with errors, check logs:

```
oc logs -n aap deployment/aap-controller-web
oc logs -n aap deployment/aap-controller-task
oc logs -n aap deployment/aap-controller-task --previous
```

Step 4: Increase Ansible Verbosity

To get more verbose output from the operator's internal automation, add `task_args` to the `AutomationController` CR spec:

```
spec:
  task_args:
    - "-vvv"
```

Apply the change and monitor the `task` pod logs.

Common OpenShift-Specific Issues

PVC not bound (wrong StorageClass)

Issue: PostgreSQL pod stays in `Pending`.
Fix: Check that the `StorageClass` specified in the CR exists and supports dynamic provisioning:

```
oc get pvc -n aap
oc get storageclass
```

Route not created (missing wildcard DNS)

Issue: AAP UI is unreachable.
Fix: Verify that your OCP cluster has a wildcard DNS entry for the router domain and that the `Route` resource was created:

```
oc get route -n aap
```

Image pull failures (missing pull secret)

Issue: Pods show `ImagePullBackOff`.
Fix: Create a pull secret for `registry.redhat.io` and link it to the namespace service account:

```
oc create secret docker-registry rh-registry-secret \
  --docker-server=registry.redhat.io \
```

```
  --docker-username=<rh-username> \
  --docker-password=<rh-password> \
  -n aap
oc secrets link default rh-registry-secret --for=pull -n aap
```

Resource quota exceeded

Issue: Pods are `Pending` with `exceeded quota` in events.
Fix: Check namespace quota and either increase it or reduce resource requests in the CR spec:

```
oc describe quota -n aap
```

Execution node not connecting to controller

Issue: Instance shows `Unavailable` in the Instances page.

Ensure port 27199 (default Receptor listener port) is open between the execution node and the OCP cluster ingress. Verify TLS certificates match by re-downloading and re-running the install bundle.

Summary

In this chapter we learned the following about deploying Automation Mesh on OpenShift Container Platform:

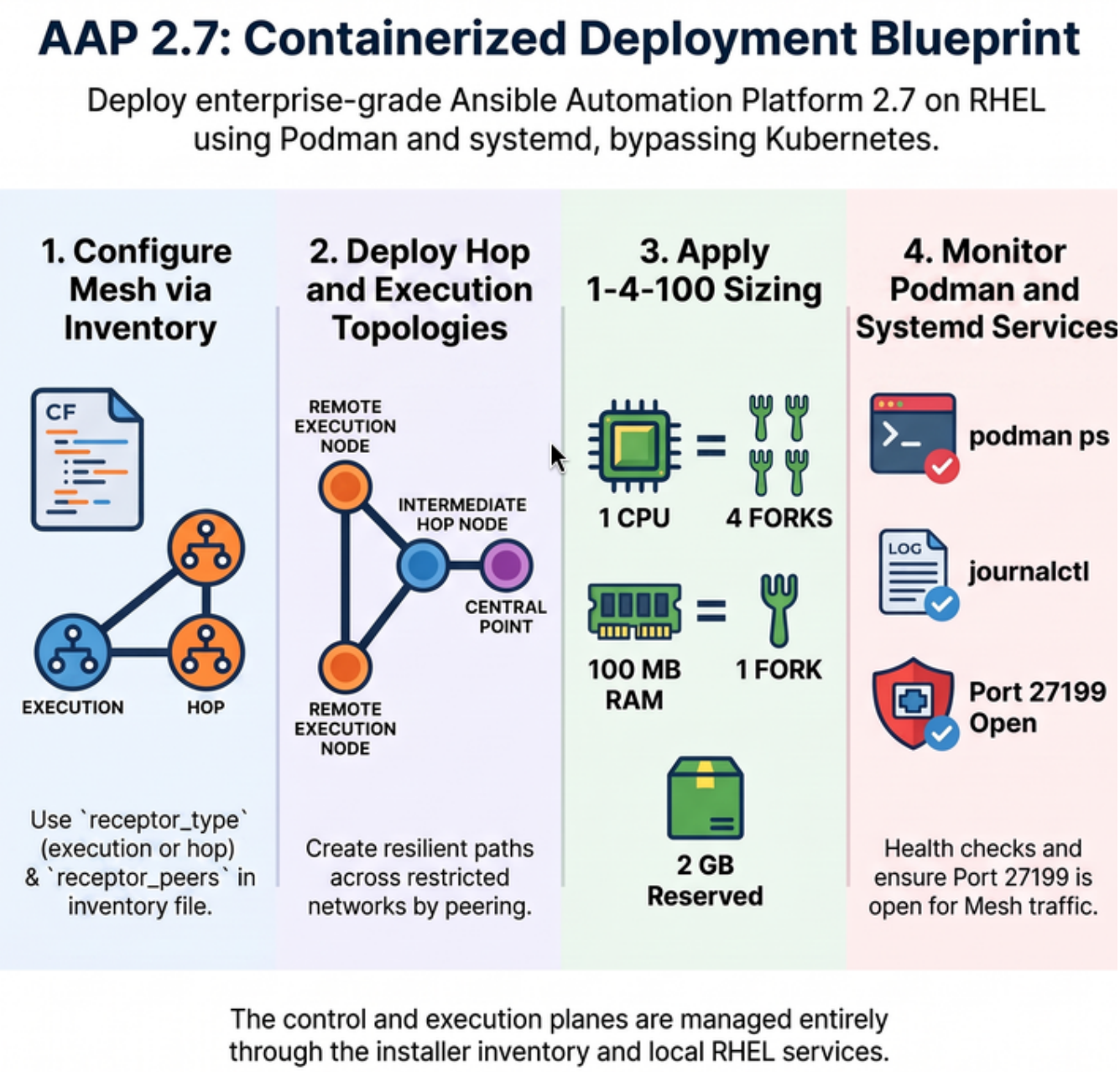
- In operator-based deployments there are **no Hybrid or Control nodes** — the Control Plane is made up of pods managed by the AAP Operator running inside the Kubernetes cluster.
- **Container Groups** are the in-cluster execution mechanism: jobs run as ephemeral pods provisioned on-demand, using a service account with RBAC permissions to create and manage pods.
- Remote execution and hop nodes are RHEL VMs peered to the OCP-hosted controller through Receptor; they are added dynamically via the AAP UI **Instances** page without re-running the installer.
- **Mesh Ingress** (`AutomationControllerMeshIngress` CR) enables pull-based connectivity for remote nodes in networks that forbid inbound connections.
- Sizing on OCP is expressed as Pod **resource requests and limits** in the `AutomationController` CR, rather than VM memory/CPU specifications.
- The 1 CPU = 4 forks and 100 MB per fork rules still apply to remote execution nodes; Container Group pods set forks at the job template level.
- Troubleshooting uses `oc describe automationcontroller`, `oc get pods`, and `oc logs` rather than manual RPM installer steps.
- Common OCP-specific issues include PVC not bound, missing pull secrets, resource quota exhaustion, and wildcard DNS misconfiguration.
- **CPU throttling**: always set CPU `requests` equal to CPU `limits` on AAP pods. Setting a limit without an equal request causes CFS throttling even when the node has spare capacity —

monitor with `container_cpu_cfs_throttled_seconds_total`.

- Live event streaming to the UI can be disabled with `UI_LIVE_UPDATES_ENABLED=false` to reduce controller load under heavy workloads; event rate and buffer settings further tune the trade-off between live visibility and system load.
- Job types (standard, sliced, workflow, system) have different capacity impacts — size the task pod based on your actual workload mix, not just concurrent job count.
- Use key performance indicators (API latency > 2s, sustained CPU > 80%, 502/503/429 error codes) to determine whether to scale web pods horizontally or the task pod vertically.
- Increase the OpenShift Route timeout (`haproxy.router.openshift.io/timeout`) on all AAP routes when clients report 504 errors on long-running operations.

Advanced Deployment of Ansible Automation Platform 2.7 — Containerized

The containerized installation of Ansible Automation Platform 2.7 runs all AAP components as containers managed by `systemd` and `podman` on Red Hat Enterprise Linux hosts, without requiring a Kubernetes or OpenShift cluster. This chapter covers Automation Mesh configuration, capacity planning, and troubleshooting specific to containerized deployments.



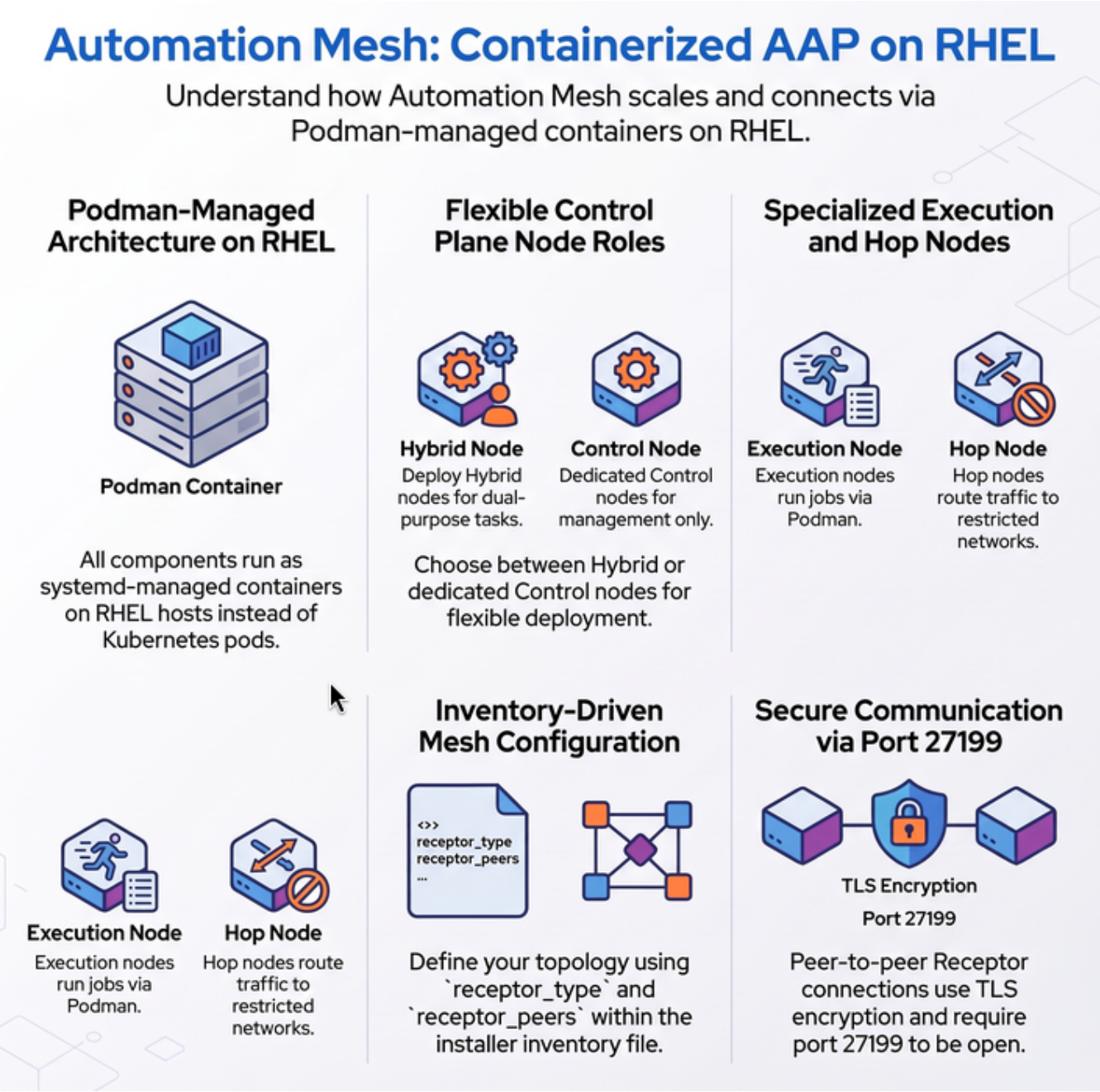
Participants will acquire practical skills in:

- Configuring Automation Mesh for a containerized AAP deployment using `receptor_type` in the installer inventory.
- Designing and deploying hop and execution node topologies using inventory peering.

- Calculating sizing requirements using the fork capacity model for RHEL-based containers.
- Troubleshooting common containerized installation issues.

Automation Mesh on Containerized AAP

Automation Mesh in a containerized deployment uses the same fundamental architecture as other deployment types — peer-to-peer Receptor connections, TLS encryption, and a Control/Execution Plane separation — but all components run as `podman`-managed containers on RHEL hosts rather than as Kubernetes pods or traditional RPM services.



Control Plane in a Containerized Deployment

The Control Plane in a containerized deployment consists of **Hybrid nodes** and **Control nodes**, just as in a VM/RPM deployment. These nodes run the persistent AAP services: the web server, task dispatcher, project updates, and management jobs.

- **Hybrid nodes** (default): Perform both control functions (managing jobs, web UI) and execution functions (running playbooks). This is the default `node_type` for nodes in the `[automationcontroller]` group.
- **Control nodes**: Run project and inventory updates and system jobs only. Execution capabilities are disabled. Set `node_type=control` to configure a control-only node.

Unlike operator-based deployments, containerized deployments **do** support Hybrid and Control nodes. There are no Container Groups in a containerized deployment.

Execution Plane in a Containerized Deployment

The Execution Plane in a containerized deployment consists of execution and hop nodes. These run as containers on RHEL hosts:

- **Execution nodes**: Run jobs using `ansible-runner` with `podman` isolation. Configured in the `[execution_nodes]` group with `receptor_type=execution` (the default).
- **Hop nodes**: Route Receptor traffic between nodes. Configured with `receptor_type=hop`. Hop nodes cannot execute automation.

In a containerized deployment, use `receptor_type` instead of `node_type` to set execution vs. hop. `node_type` was used in RPM-based deployments only.

Peers in a Containerized Deployment

Peer relationships are defined in the installer inventory using `receptor_peers`. The value must be a **comma-separated list of hostnames** — not inventory group names. By default, nodes in the `[execution_nodes]` group are automatically peered to nodes in the `[automationcontroller]` group. You can override this with `receptor_peers`.

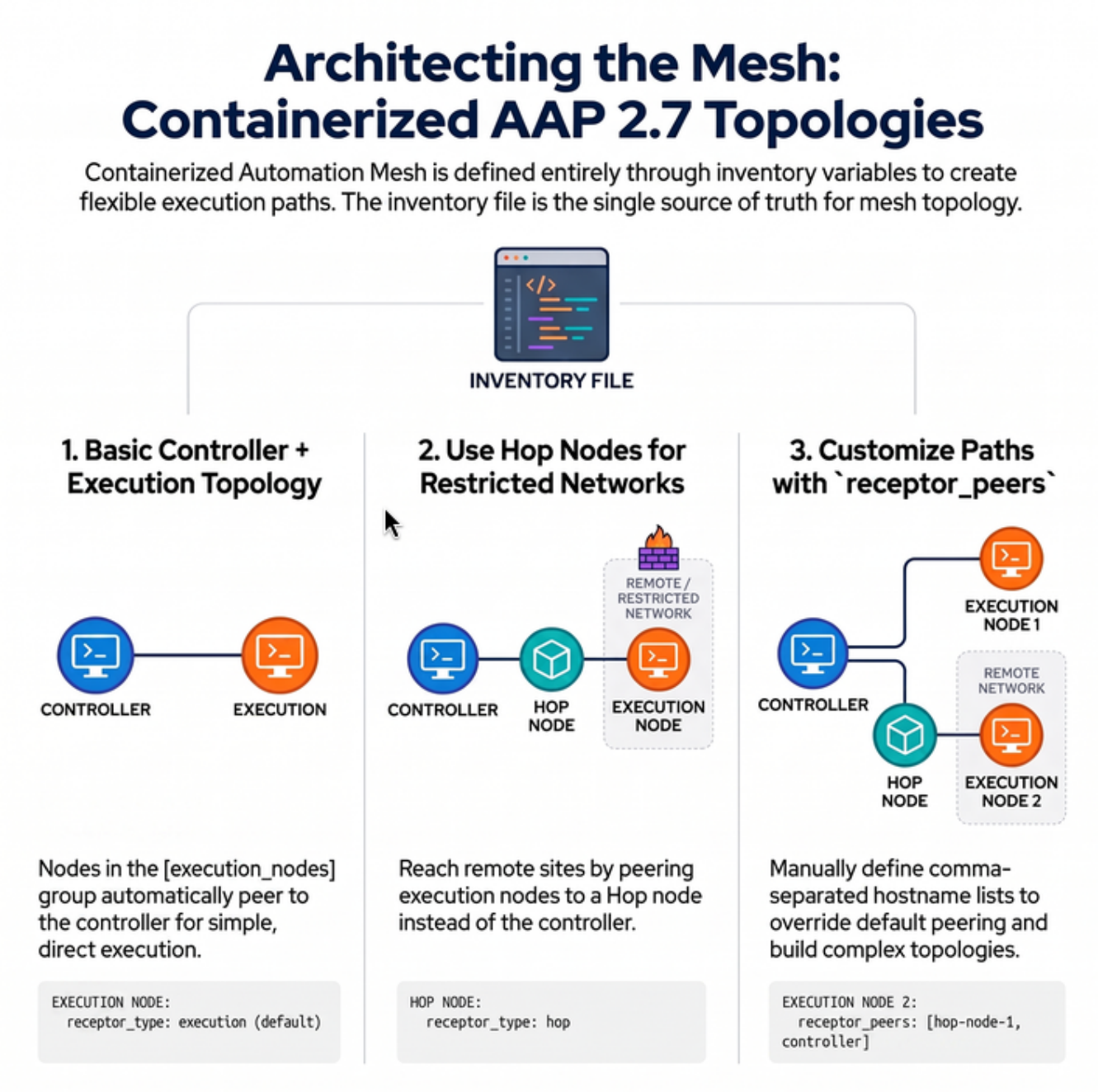
Key Differences from OCP Deployment

Feature	Containerized	OCP (Operator-based)
Control Plane nodes	Hybrid and Control nodes (containers on RHEL)	Kubernetes pods (no hybrid/control nodes)
Execution mechanism	`ansible-runner` with `podman` on RHEL	Container Groups (ephemeral pods) or remote RHEL execution nodes
Node type config	`receptor_type` in inventory file	AAP UI Instances page
Peer config	`receptor_peers` in inventory file	AAP UI (Peers from control nodes toggle)
Scaling	Re-run installer with updated inventory	Add/remove from UI without re-running installer

Automation Mesh Examples for Containerized Deployment

Let's take a closer look at Automation Mesh topology examples in a containerized AAP 2.7 installation. Mesh configuration is done entirely in the installer inventory file using `receptor_type` and `receptor_peers`.

In containerized deployments, replace `node_type` with `receptor_type`. The values are the same: `execution` (default) and `hop`.



Example 1: Basic Controller + Execution Node

A minimal containerized deployment with one controller (acting as hybrid) and one execution node.

```
[automationgateway]
aap-gateway.example.com

[automationcontroller]
aap-controller.example.com

[execution_nodes]
aap-execution.example.com

[automationhub]

[automationeda]

[database]
aap-database.example.com

[all:vars]
postgresql_admin_username=postgres
postgresql_admin_password=redhat

bundle_install=true
bundle_dir='{{ lookup("ansible.builtin.env", "PWD") }}/bundle'

redis_mode=standalone

gateway_admin_password=redhat
gateway_pg_host=aap-database.example.com
gateway_pg_password=redhat

controller_admin_password=redhat
controller_pg_host=aap-database.example.com
controller_pg_password=redhat
```

In this configuration, nodes in `[execution_nodes]` are automatically peered to nodes in `[automationcontroller]`. The controller acts as a hybrid node by default.

Example 2: Controller + Hop Node + Execution Node

A more advanced topology where an execution node in a remote or restricted network is reached via a hop node.

```
[automationgateway]
aap-gateway.example.com

[automationcontroller]
aap-controller.example.com

[execution_nodes]
aap-hop.example.com receptor_type=hop
```

```
aap-execution-remote.example.com receptor_peers=aap-hop.example.com

[automationhub]

[automationeda]

[database]
aap-database.example.com

[all:vars]
postgresql_admin_username=postgres
postgresql_admin_password=redhat

bundle_install=true
bundle_dir='{{ lookup("ansible.builtin.env", "PWD") }}/bundle'

redis_mode=standalone

gateway_admin_password=redhat
gateway_pg_host=aap-database.example.com
gateway_pg_password=redhat

controller_admin_password=redhat
controller_pg_host=aap-database.example.com
controller_pg_password=redhat
```

Here, `aap-hop.example.com` is configured as a hop node (`receptor_type=hop`). The remote execution node `aap-execution-remote.example.com` peers to the hop node using `receptor_peers`. The hop node itself is automatically peered to the controller.

The concept behind Execution and Hop nodes is the same as in an RPM deployment, but the configuration uses `receptor_type` instead of `node_type`, and `receptor_peers` (a comma-separated hostname list) instead of `peers`.

Example 3: Multiple Execution Nodes with Separate Peering

```
[automationcontroller]
aap-controller.example.com

[execution_nodes]
aap-hop.example.com receptor_type=hop
aap-execution1.example.com
aap-execution2.example.com receptor_peers=aap-hop.example.com
```

In this example:

- `aap-execution1.example.com` peers directly to the controller (default behavior).
- `aap-execution2.example.com` peers to the hop node, which routes traffic back to the controller.

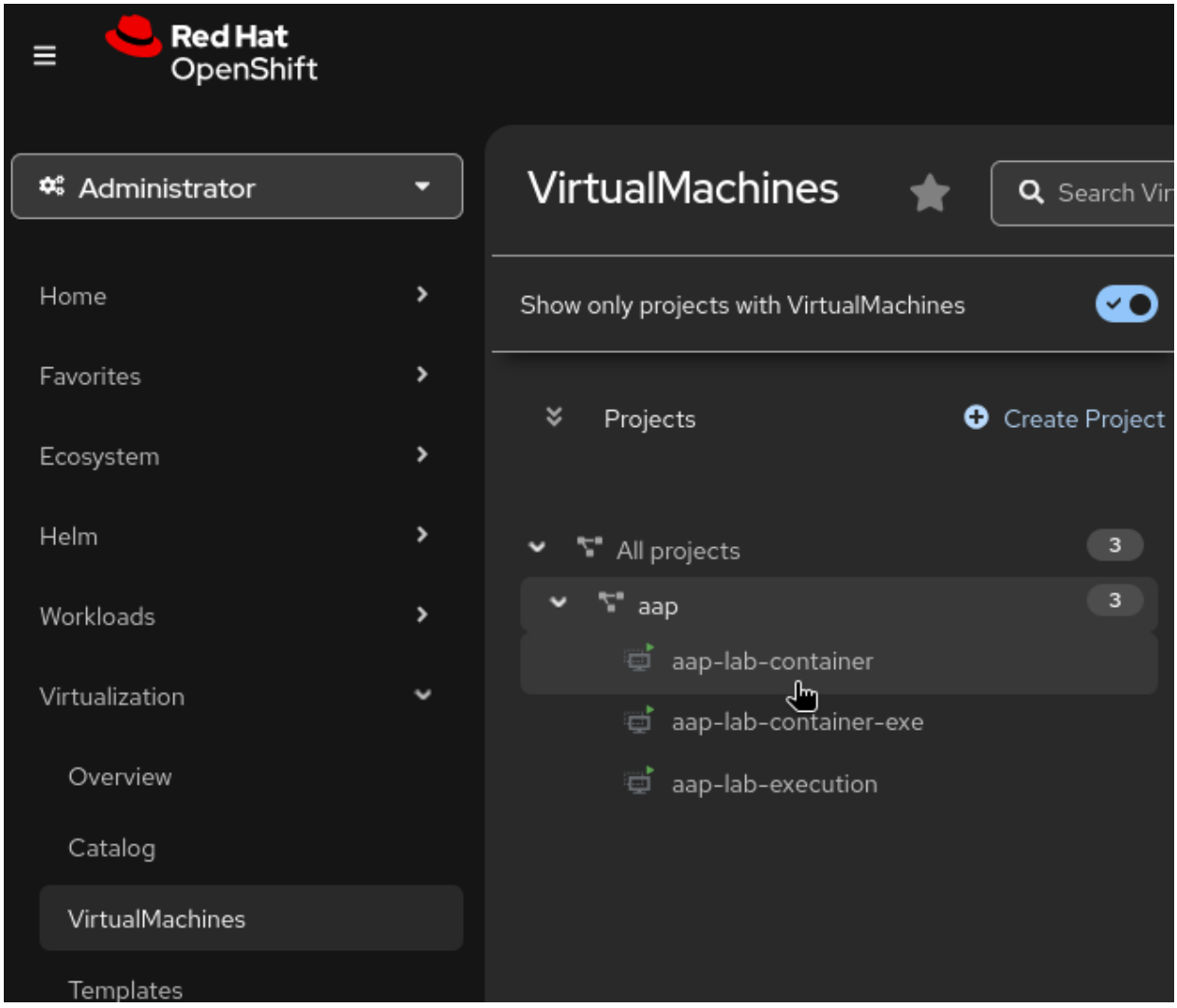
- `aap-hop.example.com` is automatically peered to `[automationcontroller]`.

For more information on design patterns refer to Automation mesh for containerized and VM environments.

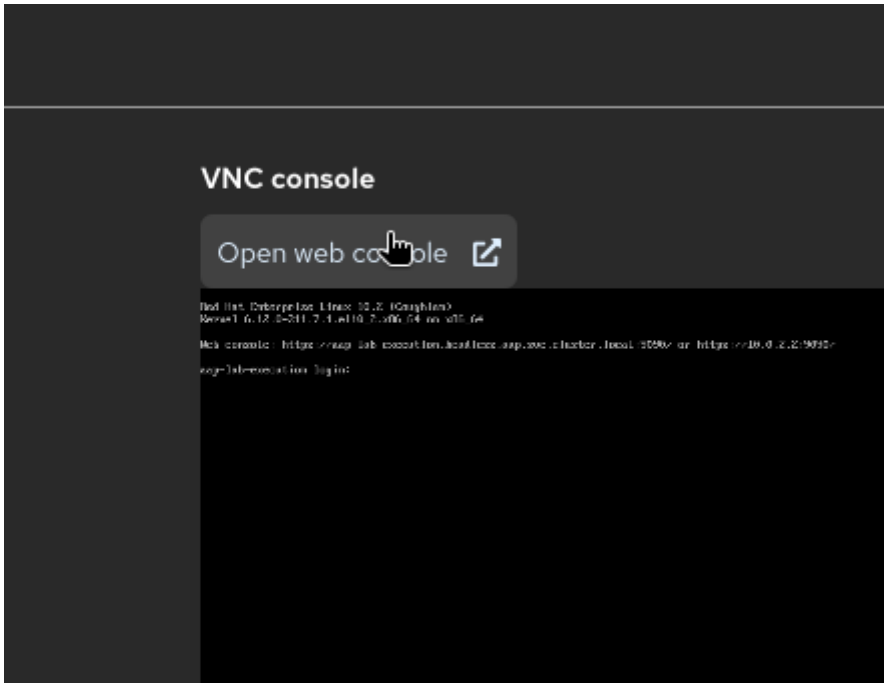
Automation Mesh Hands-on Lab (Containerized)

The following steps demonstrate how to install all AAP 2.7 components on a single system using the all-in-one pattern and adding the execution environment to the mesh. The lab will also demonstrate how to configure the execution node and verify that it is peered with the controller.:

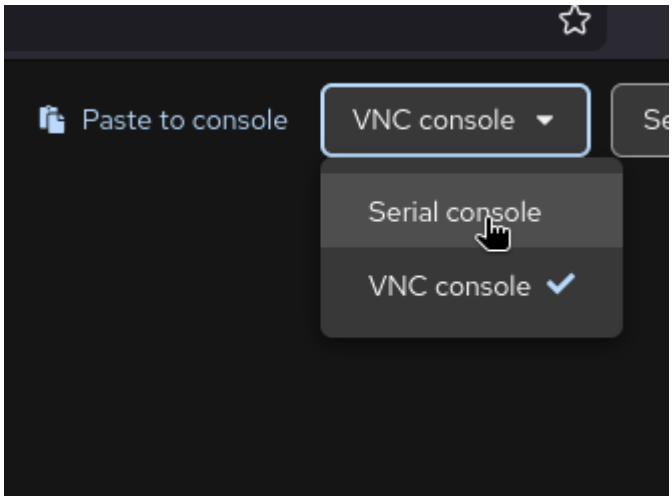
1. **OpenShift Console Access:**
    a. Open the OpenShift Console at `{openshift_console_url}` and log in with username `{openshift_username}` and password `{common_admin_password}`.
    b. Navigate to **Virtualization → VirtualMachines**
    c. Select namespace: `aap-lab`
    d. Click on `aap-lab-container`



    e. Click on the **Open Web Console** button.



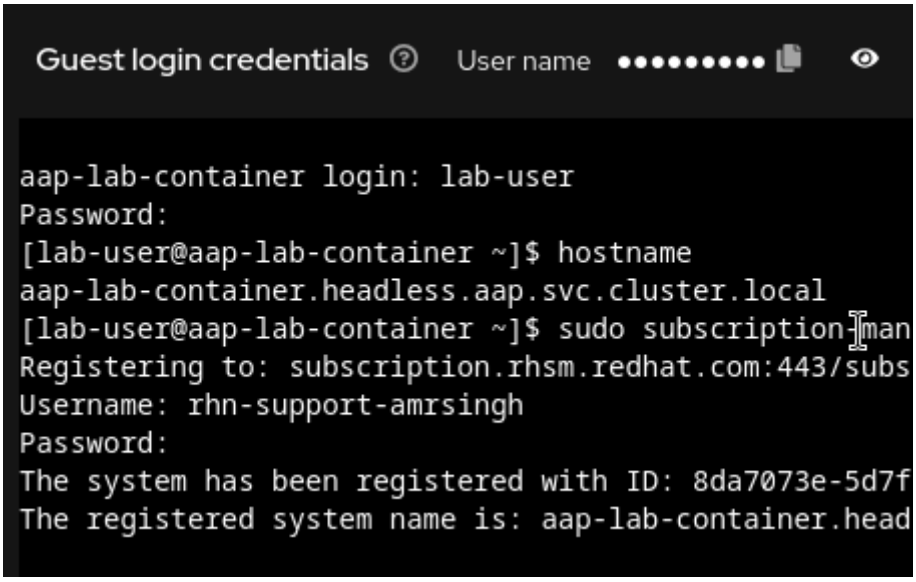
f. Use the **Serial** tab for terminal access.



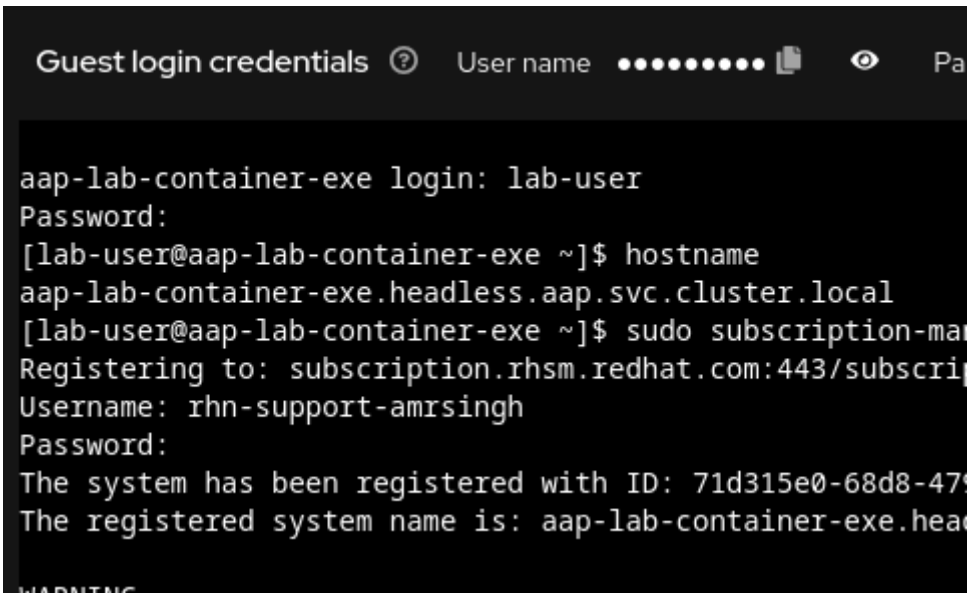
2. Log in to the VM using username `{bastion_ansible_user}` and password `{common_user_password}`. Copy and paste with CTRL+SHIFT+V in the terminal.
3. Register the system with Red Hat Subscription Management and attach the subscription to the system.

```
sudo subscription-manager register
```

```
username: <YOUR-RHN_LOGIN>
password: <YOUR-RHN_PASSWORD>
```



Follow the above steps for the aap-lab-container-exe as well once the you will register the system with Red Hat Subscription Management and attach the subscription to the system after that you can proceed with the below steps for the `aap-lab-container`.

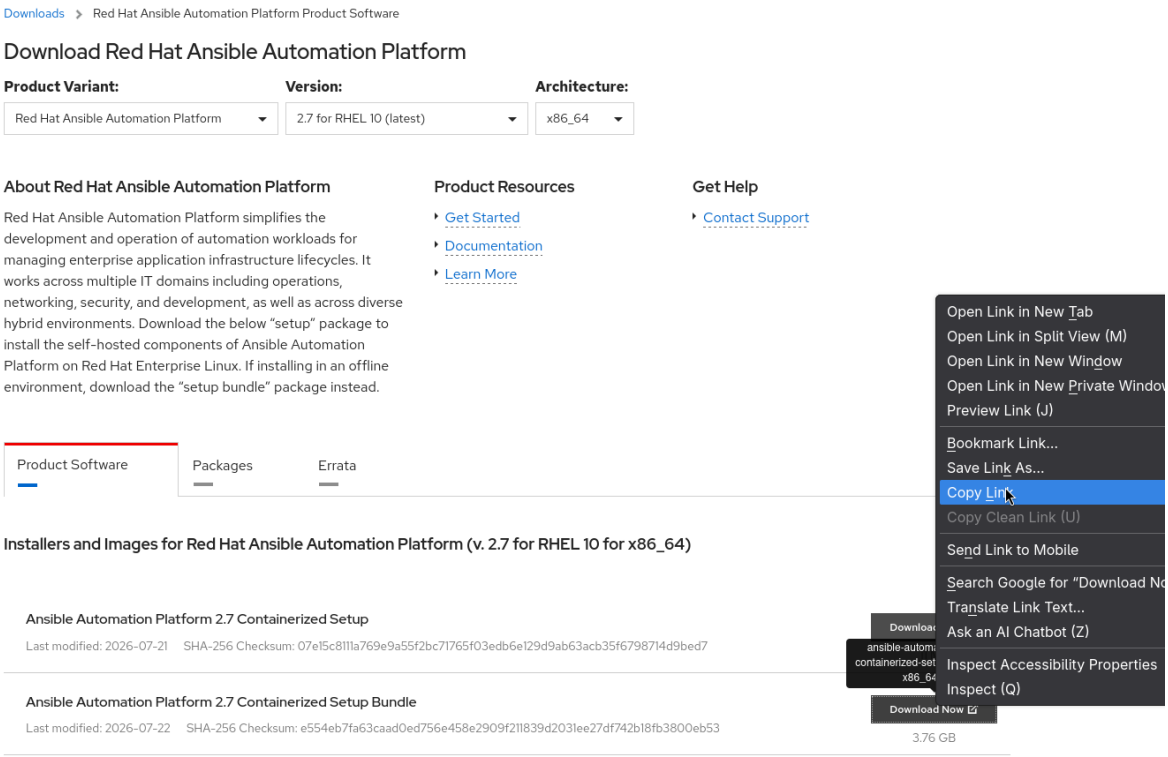


4. Install the required and useful utilities:

```
sudo dnf install -y wget git-core rsync vim ansible-core sshpass
```

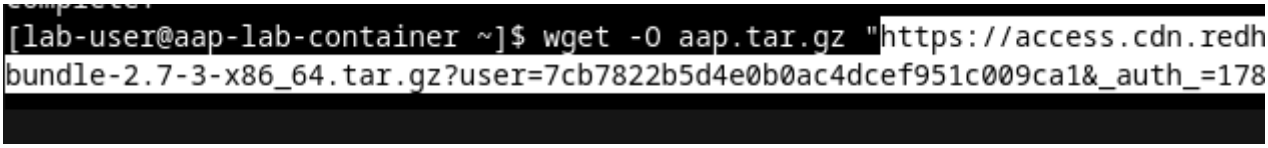
5. Go to the Installer link and Get the installer link for **Ansible Automation Platform 2.7 Containerized Setup Bundle** for RHEL 10.

   Note: Please follow steps 5 and 6 one after another in order to avoid the link expiration error, as the copied URL has an expiration time. Reload the website, repeat step 5, and copy the link if you are having trouble.



6. Download the installer file to the VM:

```
wget -O aap.tar.gz "<paste-link-here taken from step 5>"
```



7. Extract the **aap.tar.gz** file using the following command:

```
tar -xvf aap.tar.gz
```

8. Get the hostname from the system and copy it.

```
hostname -f
```

Password security guidelines:

- Use strong, unique passwords for each component.
- Do NOT use special characters like `,`, `"`, or `@` in passwords — they can break the installer.
- Supported special characters: `!`, `#`, `$`, `%`, and `&`.
- Store the inventory file securely after installation — it contains sensitive credentials.

9. Using the **cd** command, enter the extracted folder. Then, use the **vim** command, open the **inventory-growth** file and make the following changes, wherever they appear throughout the entire **inventory-growth** file:

```
 cd <extracted-folder-from-step-6>
 vim inventory-growth
aap.example.org → < paste system hostname>
<set your own> → redhat

<set your own> ← This value can be customized as per your need but remember to use
the same while doing the login after the deployment.
```

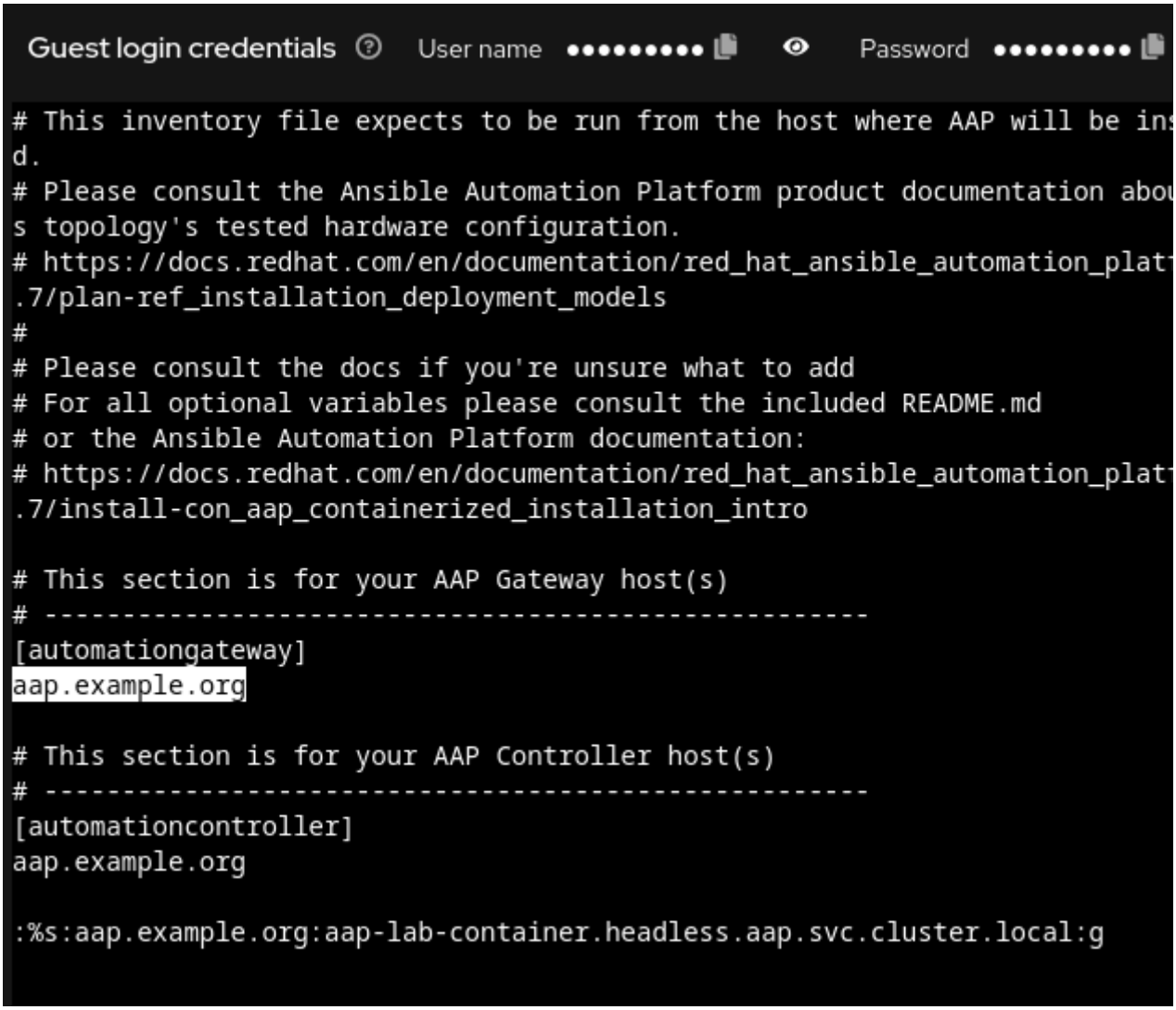
To replace all occurrences at once, use vim's global substitute command instead of editing each line manually. While in vim, type the following in **command mode** (press **Esc** first):

```
# Replace every occurrence of aap.example.org with your actual hostname
:%s/aap.example.org/<your-hostname>/g

# Replace every occurrence of <set your own> with your chosen password
:%s/<set your own>/redhat/g
```

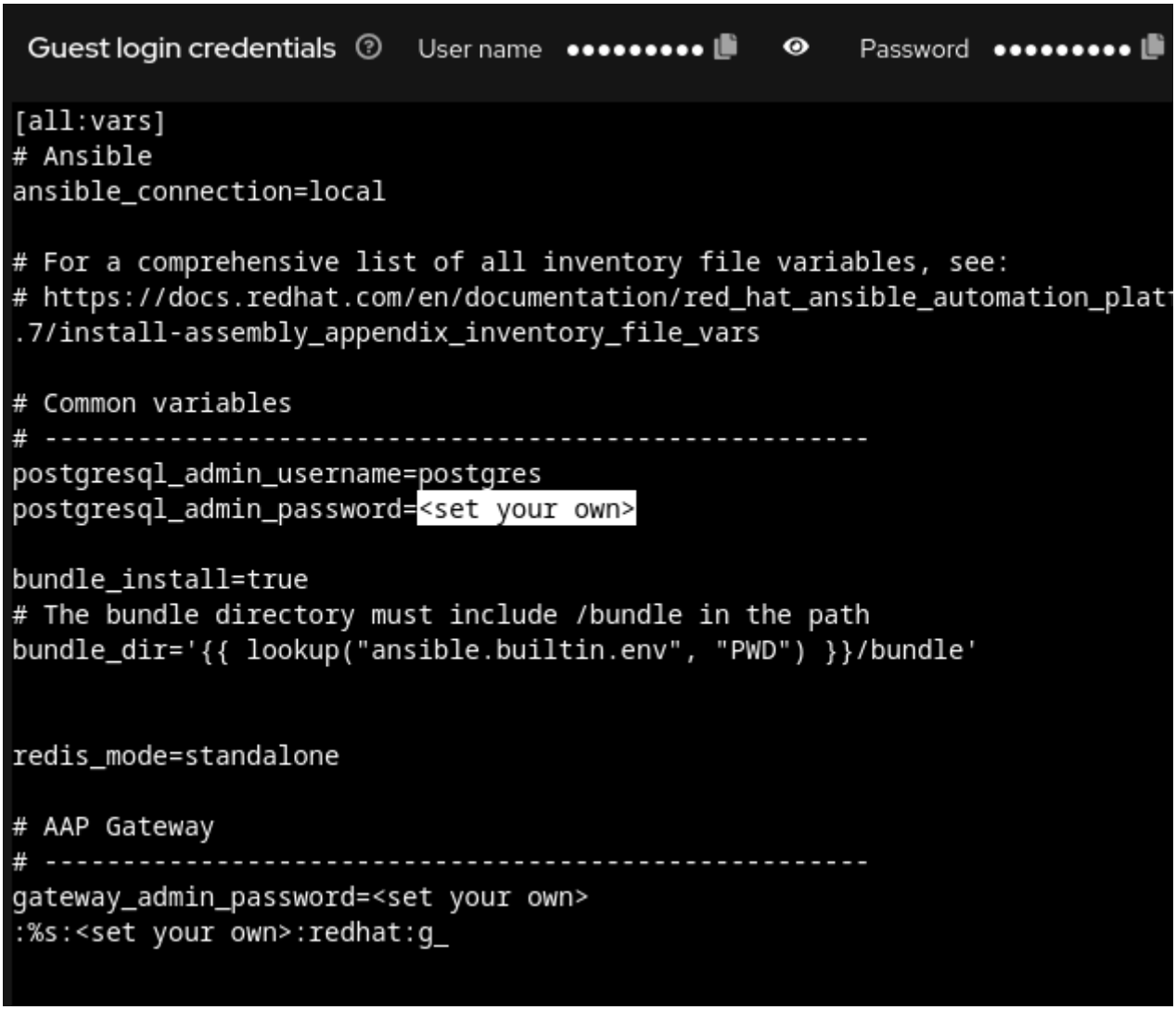
- `:%s` — runs the substitution across **all lines** in the file.
- `/g` — replaces **every match on a line**, not just the first.
- After running both commands, type `:wq` and press **Enter** to save and quit.
- If you chose a different password instead of `redhat`, use that value in the second command. Use the same password when logging in after deployment.

Click this link to learn how to use Vim basic operations.


Guest login credentials
User name
Password
This inventory file expects to be run from the host where AAP will be ins
d.
Please consult the Ansible Automation Platform product documentation abou
s topology's tested hardware configuration.
https://docs.redhat.com/en/documentation/red_hat_ansible_automation_platf
.7/plan-ref_installation_deployment_models
#
Please consult the docs if you're unsure what to add
For all optional variables please consult the included README.md
or the Ansible Automation Platform documentation:
https://docs.redhat.com/en/documentation/red_hat_ansible_automation_platf
.7/install-con_aap_containerized_installation_intro
This section is for your AAP Gateway host(s)

[automationgateway]
aap.example.org
This section is for your AAP Controller host(s)

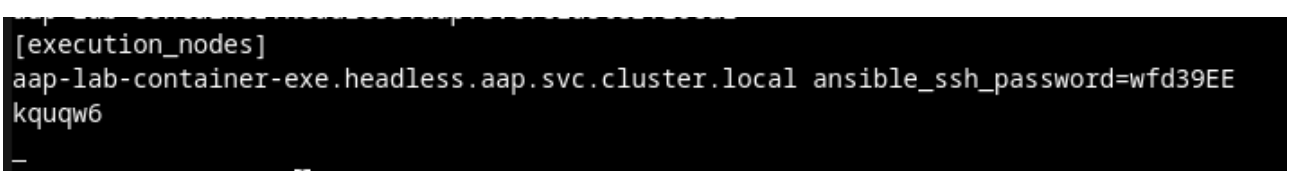
[automationcontroller]
aap.example.org
:%s:aap.example.org:aap-lab-container.headless.aap.svc.cluster.local:g



10. Now after that do minor changes to use the execution node in the mesh. In the `[execution_nodes]` section, add the following line to specify the execution node's hostname and the `ansible_ssh_password` is `{common_user_password}` :

```
vim inventory-growth
```

```
[execution_nodes]
aap-lab-container-exe.headless.aap.svc.cluster.local
ansible_ssh_password=`{common_user_password}`
```




```
[all:vars]
# Ansible
#ansible_connection=local <-- remove or comment this line
```

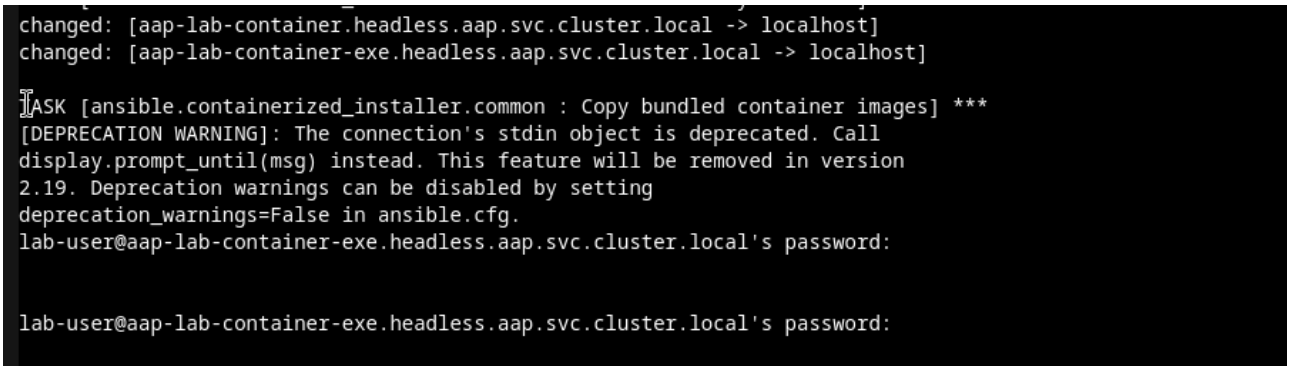
```
[database]
aap-lab-container.headless.aap.svc.cluster.local

[all:vars]
# Ansible
#ansible_connection=local
```

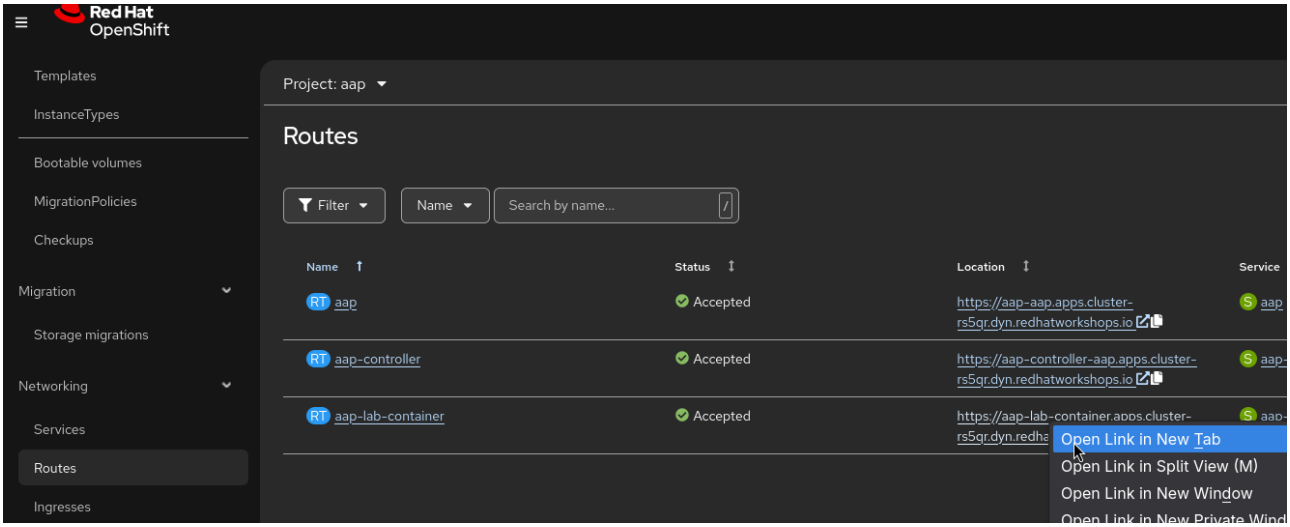
11. Once the inventory file is ready follow the below steps to do the installation and pass the `-k` option to prompt for the SSH password for the nodes in the inventory file. The password is `{common_user_password}`.:

```
ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory-growth
ansible.containerized_installer.install -k
```

12. During the installation, you will be prompted to enter the SSH password for the nodes Enter `{common_user_password}` when prompted.

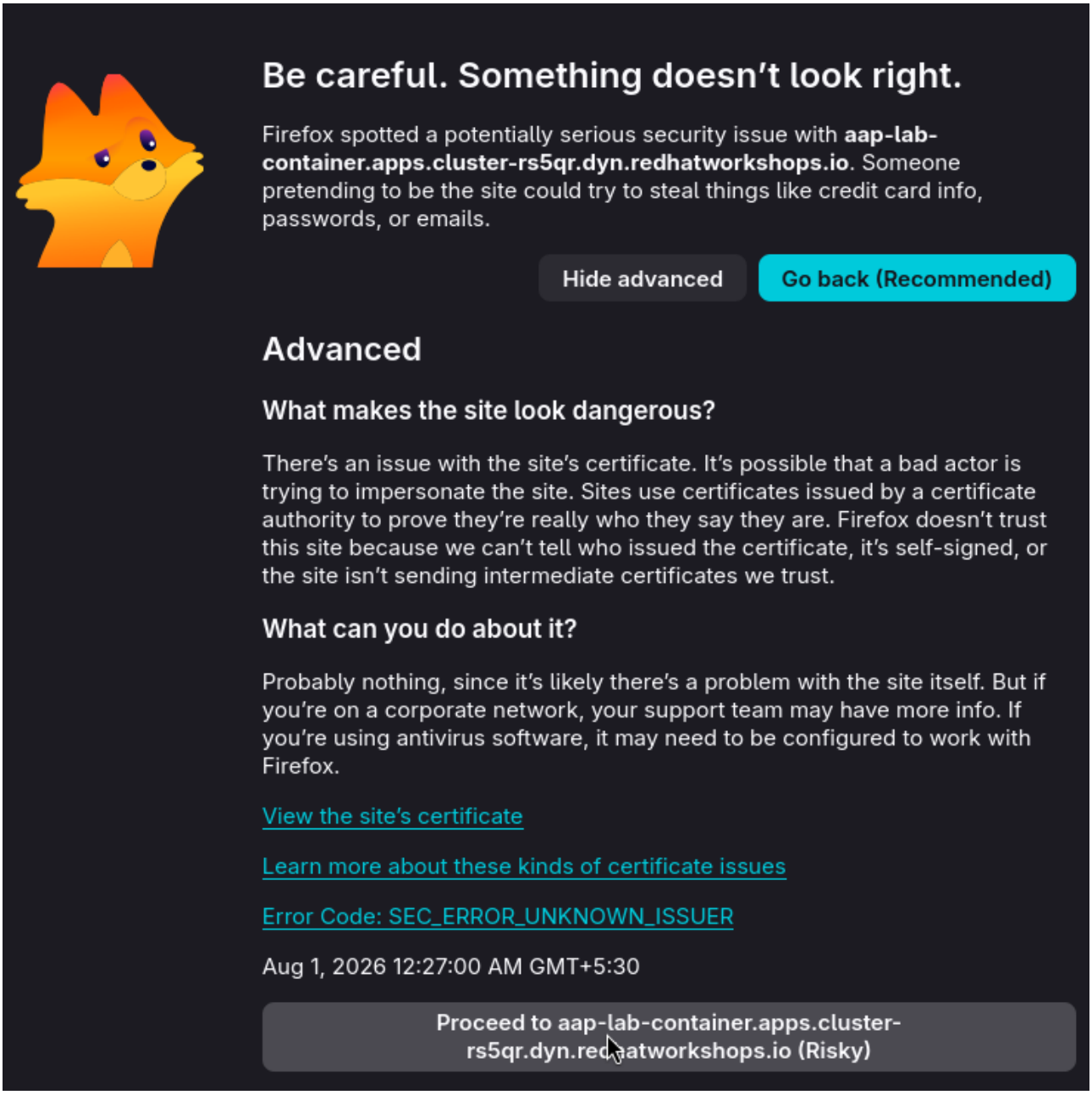


13. Go to Openshift Console and click on the Networking → Routes and click on the **aap.example.org** route to get the URL for accessing the AAP web interface. Copy the URL and open it in a new browser tab.

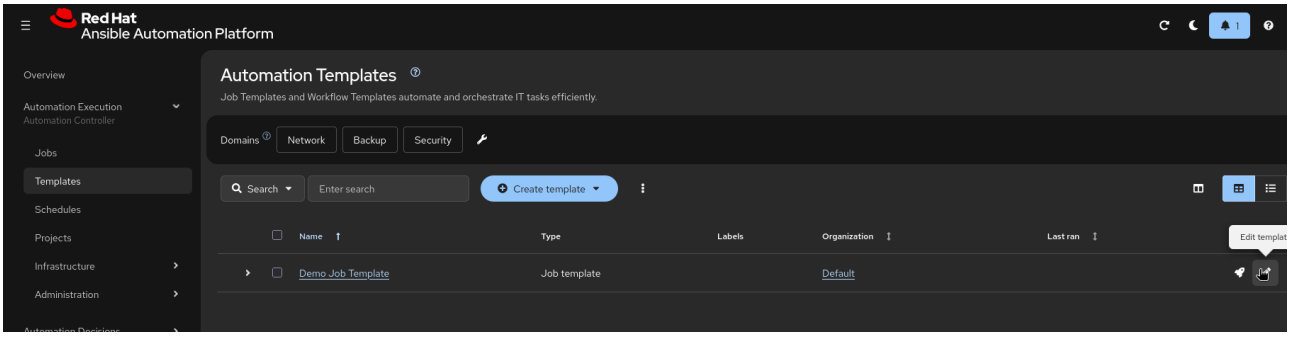


14. Use the user **admin** and the **password** set during the time of installation. The example above uses `redhat` as the password.

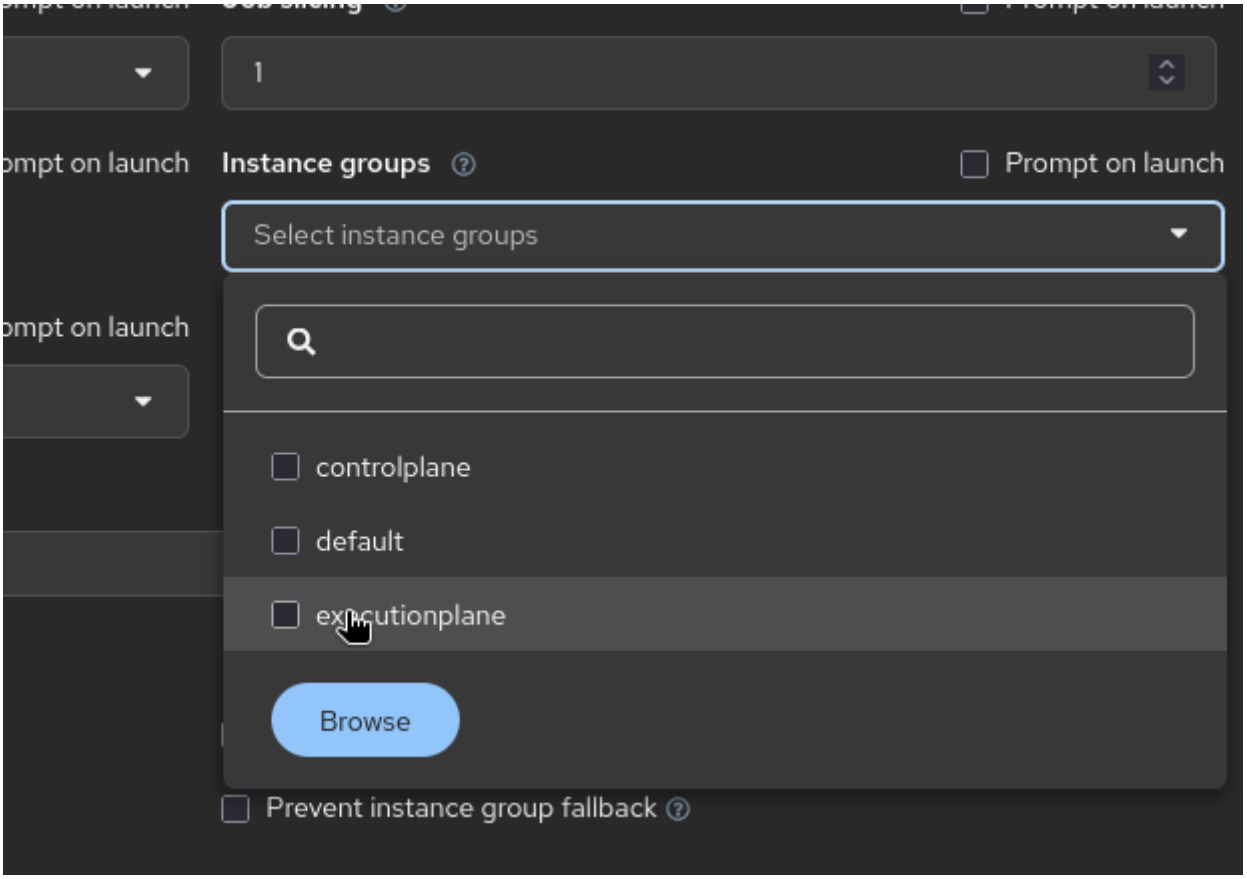
Use a stronger password for enterprise deployments.

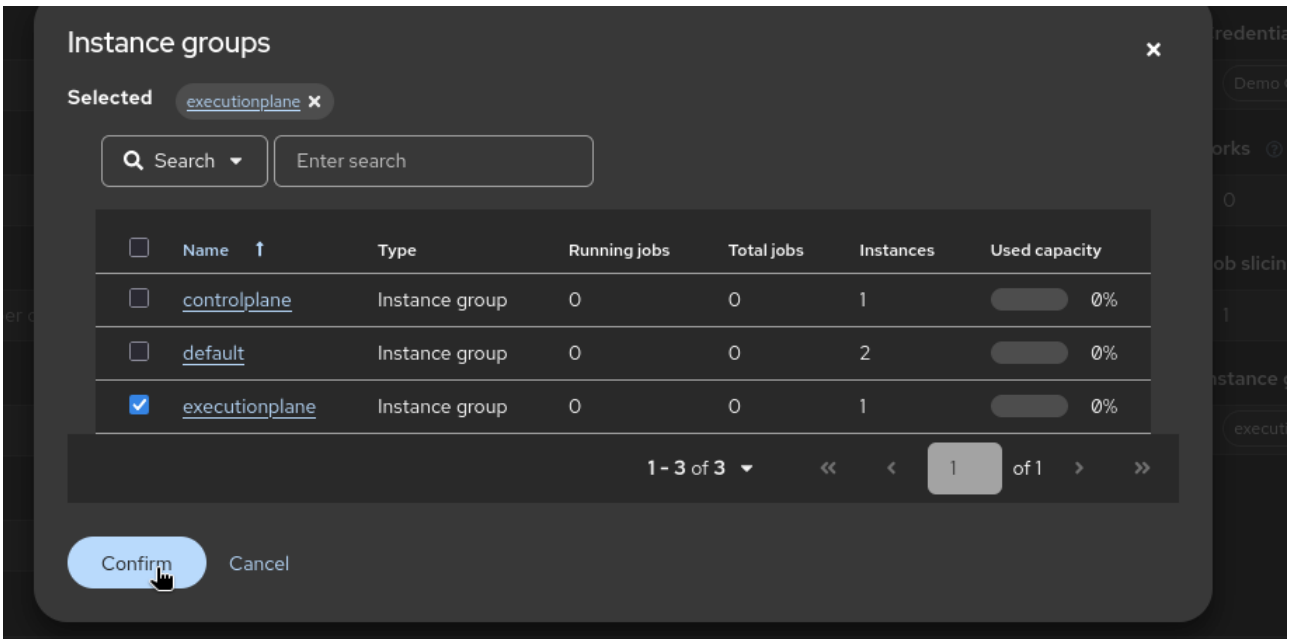


15. Select the Username and Password fields and provide the **Red Hat login ID** to get the subscription for attaching it to the AAP and **60-day subscription to Red Hat Ansible® Automation Platform** or **Red Hat Developer Subscription for Individuals** subscription can be used for this lab.
16. You can access the AAP web interface using the **admin** user and the **password** set according to the Step 9. and use the link given in the RHDP login page: **After AAP Installation** to access the AAP web interface.
17. In **Automation Execution** click on **Templates** and edit the **Demo Job Template**.

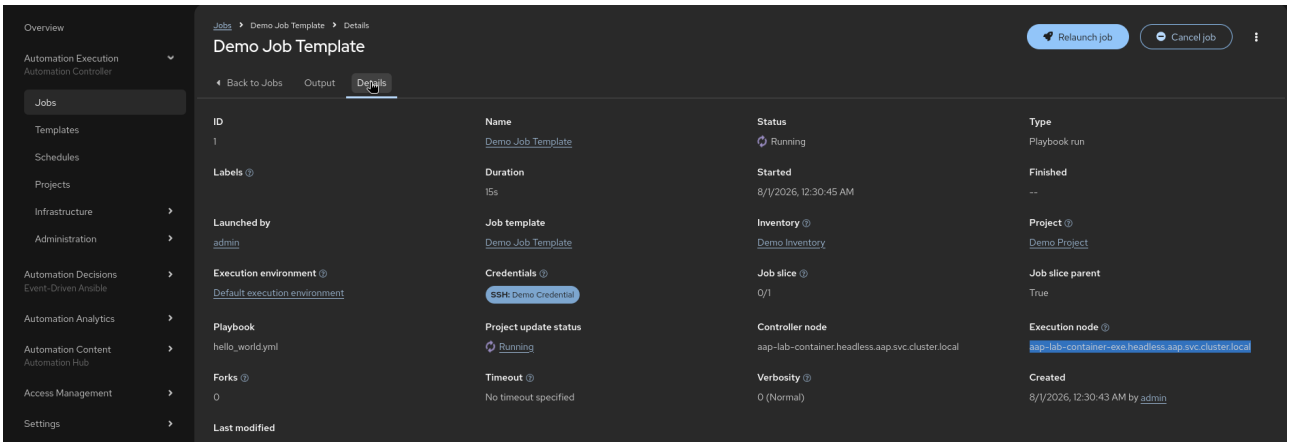


18. Click on **Instance groups** and select **executionplane**, then save the job template after selecting the execution node.





19. Run the **Demo Job Template**.
20. Go to **Jobs**, select **Demo Job Template**, and check the **Details** section to verify the job executed on the Execution Plane node.



21. Go to **Automation Execution** > **Infrastructure** > **Topology View** to confirm the current mesh architecture showing the controller peered to the execution node.

Use a stronger password than `redhat` in production deployments.

Calculate Sizing Requirements for Containerized Deployment

Having the right amount of resources for Ansible Automation Platform helps make the platform reliable and reduces cost. In a containerized deployment, AAP components run as containers on RHEL hosts managed by `podman` and `systemd`. Sizing is expressed in terms of host VM (or bare-metal) CPU, memory, and disk — but the capacity calculations are the same as for traditional VM deployments.

The high-level points to consider during capacity planning:

- Characterizing the workloads.
- Reviewing the capabilities of different node types.
- Planning the deployment based on workload requirements.

Isolated factors that play a major role in calculating sizing:

- Number of managed hosts controlled by AAP.
- Jobs or tasks running per hour per host.
- Maximum number of concurrent jobs to support.
- Maximum number of forks set on jobs.
- Maximum API requests per second.
- Node size preferred for deployment (CPU/Memory/Disk).

Unified UI / Automation Gateway

The Automation Gateway provides authentication and load-balancing of requests using the gRPC service. Increasing CPU and memory reduces bottlenecks under load. If you see 502 gateway errors during access attempts, the gateway may be a bottleneck.

Automation Controller (Control Plane)

In a containerized deployment, the Control Plane is made up of **Hybrid nodes** (default) and **Control nodes**:

- **Hybrid nodes**: Can manage jobs (control) and run playbooks (execution). Default type in the `[automationcontroller]` group.
- **Control nodes**: Manage and dispatch jobs but do not run playbooks. Set `receptor_type=control` to configure.

Key concept for control capacity:

- Each job needs 1 unit of control capacity.
- A node with 100 capacity units can control up to 100 concurrent jobs.

Vertical scaling a controller node (more CPU/RAM on the RHEL host) increases:

1. The number of jobs manageable simultaneously.
2. The speed of job event processing.

Vertically scaling a controller node does not automatically increase the number of web workers. For more web concurrency, scale **horizontally** by adding more controller nodes — a load balancer distributes requests across them. Horizontal scaling also adds redundancy.

Recommended vertical scaling practice: scale CPU and memory together in a **1 CPU to 4 GB RAM ratio**. Add CPU even when it is not a bottleneck — during high memory utilization, higher CPU processes the accumulated event queue faster.

Execution Nodes

Execution and Hybrid nodes provide execution capacity. The capacity consumed by a job running against one host equals one fork.

Vertical scaling an execution node (larger RHEL VM with more resources) provides more forks for job execution, increasing the number of concurrent jobs.

Horizontal scaling execution nodes (adding more nodes to an instance group) ensures lower-priority jobs do not block higher-priority jobs when nodes are grouped by workload.

Hop nodes use minimal resources; vertical scaling does not increase their capacity. Instead, monitor bandwidth — especially for hop nodes connecting large numbers of execution nodes. Consider a network upgrade if bandwidth approaches capacity.

Horizontal scaling of hop nodes increases redundancy and keeps traffic flowing if a hop node fails.

Memory Relative Capacity

`mem_capacity` is the maximum memory needed by one fork. The default value is 100 MB. 2 GB of memory is reserved for AAP services.

```
Example: 16 GB of available node memory → (16384 - 2048) / 100 ≈ 140 forks
```

CPU Relative Capacity

Ansible workloads are often processor-bound. The baseline is 1 CPU = 4 forks.

```
4 CPUs x 4 forks per CPU = 16 forks
```

The `capacity_adjustment` value (0 to 1) determines how close the chosen fork count is to the memory-bound maximum. At 0.5:

```
Capacity = [140 forks (memory) x 0.50] + [16 forks (CPU) x 0.50]
         = 70 + 8 = 78 forks
```

Event-Driven Ansible Controller (AutomationEDA)

EDA workloads are sized based on:

- Number of simultaneous rulebook activations.
- Number of events received per minute.

By default, EDA allows 12 rulebook activations per node and 24 total. Override with `MAX_RUNNING_ACTIVATIONS` in `/etc/ansible-automation-platform/eda/settings.yaml`. Restart the service after changing the value:

```
automation-eda-controller-service restart
```

Each rulebook activation container has a 200 MB memory limit by default. For example, with 4 CPUs and 16 GB RAM, one activation with 200 MB cannot handle more than 150,000 events per minute. To increase the limit to 400 MB, set `PODMAN_MEM_LIMIT = 400m` in `/etc/ansible-automation-platform/eda/settings.yaml` and restart the service.

Automation Hub (Private Automation Hub)

PAH provides Execution Environments and collections. Storage requirements dominate:

- Number of Execution Environments stored.

- Size and version count of each Execution Environment.

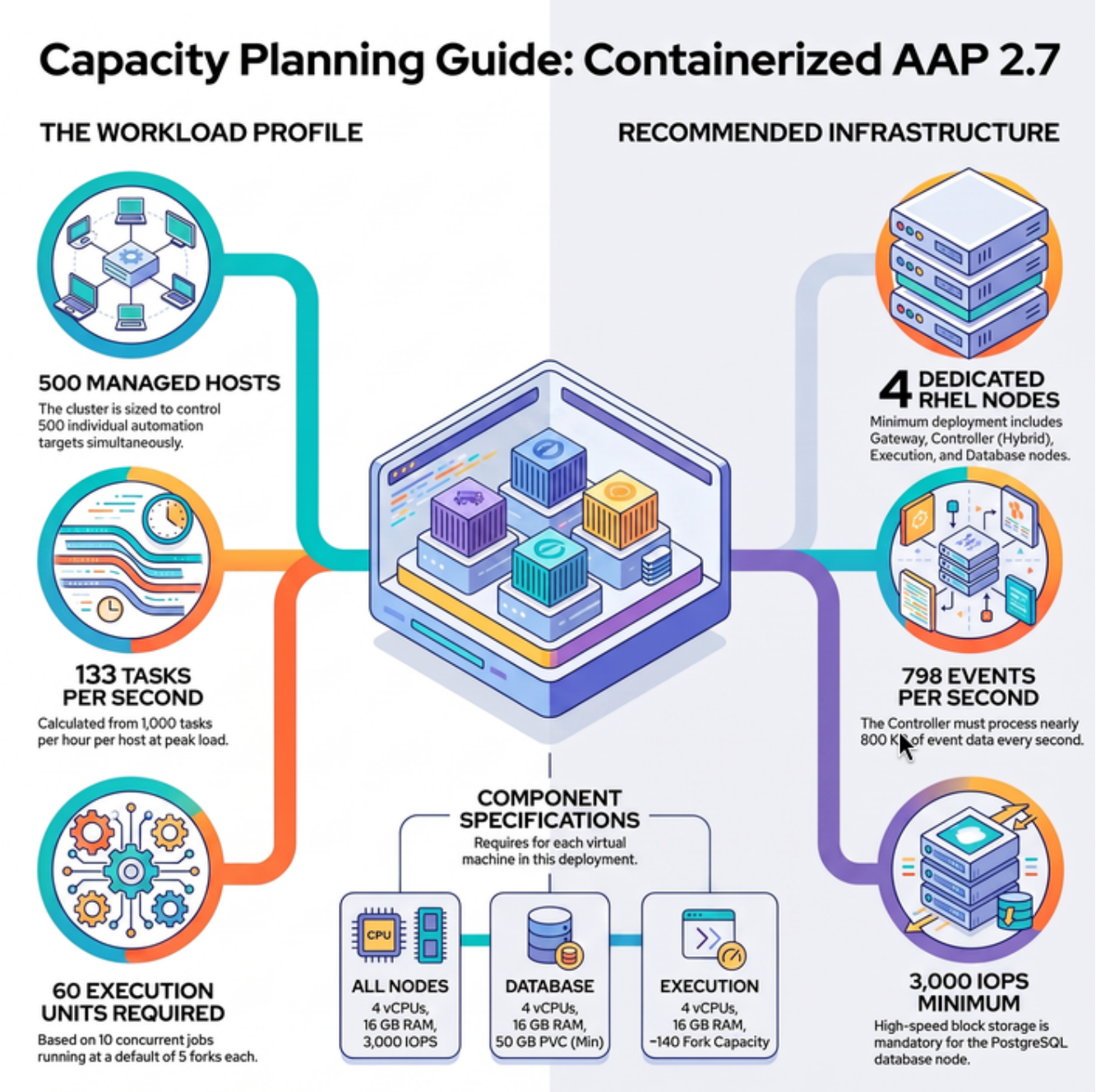
Ensure adequate disk space on the PAH host. The default database allocation is usually sufficient, but calculate precisely using the factors above for large environments.

Example: Calculating Sizing for a Containerized Deployment

At this point we should have a good idea how to calculate capacity and what factors to consider. Let's go through a planning exercise for a containerized AAP 2.7 deployment.

For this example, the cluster must support the following capacity:

- 500 managed hosts
- 1,000 tasks per hour per host (16 tasks per minute per host)
- 10 concurrent jobs
- Forks set to 5 on playbooks (the default)
- The average event size is 1 KB



Target virtual machine size: 4 CPUs and 16 GB RAM, disks with 3,000 IOPS.

Execution Capacity Calculation

Execution Capacity = (Number of jobs × forks value) + (Number of jobs × 1 base task impact)

```
(10 jobs x 5 forks) + (10 jobs x 1) = 60 execution capacity units
```

Control Capacity Calculation

```
Total tasks/min = 500 managed hosts x 16 tasks/min = 8,000 tasks/min = 133 tasks/sec
```

In other words, 133 events are generated every second.

It is a good practice to run the job first and count exactly how many events it produces — this depends on the specific task and verbosity level. For example, a debug task printing "Hello World" produces 6 job events per host at verbosity 1, and 34 at verbosity 3.

Assuming 6 events per task:

```
133 tasks/sec x 6 events/task = 798 events/sec
798 events x 1 KB/event = 798 KB/sec of log/event data
```

Our controller node must be able to process approximately 798 events per second and provide an execution capacity of 60 forks.

Given the 16 GB node we selected:

```
(16384 - 2048) / 100 = 140 forks available
140 forks > 60 required → 1 execution node is sufficient
```

Two nodes are recommended for redundancy.

Sizing Summary

To summarize, for this workload on a containerized deployment deploy the following RHEL hosts with 4 CPUs at 2.5 GHz, 16 GB RAM, and disks with ~3,000 IOPS:

- **1 Automation Gateway node** (mandatory for AAP 2.7)
- **1 Automation Controller node** (hybrid, handles both control and execution)
- **1 Execution node** (minimum; 2 recommended for redundancy)
- **1 Database node**

Automation Hub and EDA nodes should be added based on workload — start with the minimum requirements from the AAP 2.7 system requirements and scale based on observed usage.

Database Sizing

```
Database size for facts  = Facts size per host x number of hosts / 1024
                         = 50 KB x 500 / 1024 ≈ 24 MB

Database size for inventory ≈ same as facts ≈ 24 MB

Database size for jobs   = number of hosts x jobs/host/day x days to keep
                           x number of events x event size / 1024
                         = 500 x 24 x 30 x 6 x 1 KB / 1024 ≈ 2 GB (30-day retention)
```

Database RAM and CPU should match the higher of the controller or execution node resource values — in this case, 4 CPUs and 16 GB RAM.

Performance Considerations for Containerized Deployments

In the previous section we discussed how to calculate node size. The following default parameters and containerized-specific factors will also help with planning control and execution capacity.

The Tuning of the parameters needs careful consideration and should be done only after measuring the actual workload and performance of the system. The default values are based on typical workloads and may not be optimal for all environments.

Default Parameters

The following parameters likely do not need to be changed unless you know a specific value and the reason it is needed:

Parameter	Default value
Facts size per host	50 KB
Size per event	1 KB
Events per task	6–10
Memory per fork	100 MB
Forks per CPU	4
Capacity Adjustment	1.0 (select highest value)
Execution Environment average size	450 MB
Automation Controller capacity	~400 events/sec
Automation Controller RAM per event fork	10 KB
Automation Controller CPU per event fork	0.0001
Concurrent API calls per controller	100 (up to 300 allowed)
Controller/execution base job capacity per job run	1

Factors That Vary Across Environments

- Number of Ansible managed hosts/nodes.
- Number of jobs per host per day.
- Tasks run per playbook or job including imported roles/tasks/playbooks.
- Whether jobs run for a specific time period or 24×7.

Containerized-Specific Performance Considerations

Podman and systemd overhead

Containerized AAP runs each service as a `podman` container under `systemd`. There is a small overhead compared to native RPM services, but it is negligible for typical workloads. Ensure `podman` is at least version 4.x for best performance and full feature support.

Container storage driver

Podman uses `overlay` storage by default. Ensure the underlying filesystem supports `overlay` (ext4 and XFS are both supported). Avoid using `vfs` in production — it has significantly worse I/O performance.

Network performance

Receptor communication between nodes uses TCP over port 27199 by default. For hop nodes routing traffic across WAN links, monitor bandwidth closely — a hop node connecting many execution nodes over a slow link becomes a throughput bottleneck before its CPU or memory is exhausted. Use `iperf3` or similar tools to validate bandwidth before deploying a multi-hop topology.

EDA event rate limits

Each rulebook activation container is limited to 200 MB of memory by default. If event rates are very high or activations require more state, increase `PODMAN_MEM_LIMIT` in `/etc/ansible-automation-platform/eda/settings.yaml` and restart the EDA service. Increase `MAX_RUNNING_ACTIVATIONS` if you need more than the default 12 activations per node.

Disk I/O

The database is the most I/O-intensive component in any AAP deployment. Ensure the database host uses fast storage — SSDs or NVMe-backed volumes with at least 3,000 IOPS. Controller and execution nodes also need adequate I/O for project syncs and execution environment pulls from Private Automation Hub.

Tune the PostgreSQL Database

PostgreSQL performance directly affects job scheduling, event processing, and API response times. Key configuration parameters to review for production workloads — set in `/var/lib/pgsql/data/postgresql.conf`:

Parameter	Recommended starting value	Effect
`shared_buffers`	25% of total RAM	Caches frequently accessed data pages in memory; reduces disk reads
`work_mem`	64MB–256MB	Per-sort and per-hash operation memory; increase for complex queries under load

Parameter	Recommended starting value	Effect
`max_connections`	Sum of all AAP component connections + 20% headroom	Prevents connection exhaustion under load
`checkpoint_completion_target`	0.9	Spreads checkpoint I/O over a longer window, reducing write I/O spikes
`wal_buffers`	16MB	Buffers WAL writes; increase on high-write workloads

Apply changes and restart:

```
sudo systemctl restart postgresql
```

Underprovisioning `max_connections` causes `FATAL: sorry, too many clients already` errors when the automation controller, hub, and EDA each attempt to open their connection pools simultaneously. Each AAP component opens multiple database connections per process. Account for all components when setting this value.

For very high connection counts, consider deploying PgBouncer as a connection pooler in front of PostgreSQL to multiplex many application connections into fewer actual database connections.

EDA Horizontal Scaling

In a containerized AAP deployment, Event-Driven Ansible controller supports multi-node deployments that separate API handling from activation execution. This allows you to scale each function independently based on your workload, without over-provisioning a single hybrid node.

Node types

Node type	Role
`api`	Handles HTTP REST API requests and WebSocket connections from users and external systems
`worker`	Runs rulebook activations and processes event streams; does not serve API requests
`hybrid` (default)	Combines API and worker functions on a single node; sufficient for small workloads

Inventory file configuration

Define node types using the `eda_type` variable in the `[automationeda]` inventory group:

```
[automationeda]
eda-api.example.com      eda_type=api
```

```
eda-worker1.example.com  eda_type=worker
eda-worker2.example.com  eda_type=worker
```

Re-run the installer after updating the inventory to apply the configuration.

Sizing guidelines

- Size **API nodes** based on the number of concurrent users and API request rates — a single API node is typically sufficient unless you have many concurrent UI sessions.
- Size **worker nodes** based on the number of simultaneous rulebook activations and event throughput — add worker nodes when `MAX_RUNNING_ACTIVATIONS` is consistently reached.
- Separating roles allows worker nodes to be scaled independently without disrupting the API layer.

Unlike OCP-based deployments, scaling containerized EDA requires re-running the installer. Existing activations are interrupted during the operation. Plan scaling during maintenance windows.

Key Performance Indicators

Monitor these signals to determine when scaling or tuning is needed:

Indicator	Signal	Action
High API latency	Response time > 2 seconds for UI or API requests	Add controller nodes to distribute web workers; check database connection count
502 Bad Gateway	Returned by automation gateway	Scale gateway node vertically; check backend service health with `podman ps` and `systemctl status`
`FATAL: too many clients` in DB logs	PostgreSQL connection limit exceeded	Increase `max_connections` in `postgresql.conf`; consider PgBouncer connection pooling
Activation failures	Rulebook activations failing to start or being queued	Check `MAX_RUNNING_ACTIVATIONS` in EDA settings; add `worker` nodes to the EDA cluster
Receptor peer disconnected	Mesh topology shows node offline in AAP UI	Verify port 27199 is reachable; check `systemctl status ansible-automation-platform-receptor` on the affected node
High disk I/O wait	`iostat` shows > 50% iowait on the database host	Upgrade to faster storage tier; tune `checkpoint_completion_target` to 0.9

Helpful Formulas

```
Total Jobs = (Number of Hosts x Jobs per Host per Day) ÷ Hosts Required per Job
```

```
Job Duration = Job Duration per Host x CEILING(Hosts per Job ÷ Parallel Forks)

Workload Utilization = (Jobs per Day x Job Duration) ÷ Automation Hours per Day

Polling Load = (Jobs per Day x Job Duration) ÷ Polling Interval

API Utilization = (API Calls per Day x API Call Duration) ÷ Automation Hours per Day

Database size for facts  = Facts size per host x number of hosts / 1024

Database size for jobs = number of hosts x jobs per host per day x number of days
                         to keep the data x number of events x event size / 1024
```

Database RAM and CPU should match or exceed the higher of the controller or execution node resource values.

Troubleshooting Containerized AAP Installation Issues

In a containerized installation, the installer automates tasks that can also be performed manually using `podman` and `systemd`. The basic rule of thumb: if the installer fails on a task, that same task will fail when run manually — identify it and fix the root cause, then re-run the installer. This helps isolate the issue and ensures steps execute under the same user and environment.

Step 1: Increase Verbosity

Add `-vvv` to the installer `ansible-playbook` command for verbose output:

```
ansible-playbook -i inventory-growth ansible.containerized_installer.install -vvv
```

Verbosity levels:

- **Level 1 (-v)**: Task results, changed/failed details, module stdout/stderr. Useful for seeing why something failed.

```
TASK [ansible.containerized_installer.common : Set ostree-based OS fact] ****
ok: [localhost] => {"ansible_facts": {"ostree": false}, "changed": false}
```

- **Level 2 (-vv)**: More info on variables, task parameters, SSH arguments, and remote command details.

```
TASK [ansible.containerized_installer.preflight : Ensure postgresql admin password is
provided] ***
task path: .../roles/preflight/tasks/database.yml:30
ok: [localhost] => {"changed": false, "msg": "All assertions passed"}
```

- **Level 3 (-vvv)**: Connection details, exact shell commands on the remote machine, fact gathering, module file paths.

```
TASK [Gather facts] ****
<127.0.0.1> ESTABLISH LOCAL CONNECTION FOR USER: lab-user
<127.0.0.1> EXEC /bin/sh -c 'echo ~lab-user && sleep 0'
```

Step 2: Reproduce the Failing Task Manually

If a task fails in the installer, try running the equivalent `podman` or `systemd` command manually on the affected host. This confirms whether the issue is environmental (file permissions, SELinux, missing packages) rather than a bug in the installer logic.

Step 3: Read Error Messages Carefully

Most preflight errors are self-explanatory. For example:

```
TASK [ansible.containerized_installer.preflight : Preflight check - Fail if this
machine
lacks sufficient RAM.] ***
fatal: [127.0.0.1]: FAILED! => {"changed": false,
"msg": "This machine does not have sufficient RAM to run Ansible Automation
Platform."}
```

This error indicates insufficient memory on the target host. Increasing RAM resolves it. The same pattern applies to disk space, CPU count, and OS version preflight checks.

Step 4: Check Container and Service Status

After installation, verify all containers are running:

```
sudo podman ps --all
```

Check systemd service status for individual components:

```
sudo systemctl status automation-controller-web
sudo systemctl status automation-controller-task
sudo systemctl status automation-gateway
sudo systemctl status automation-eda
```

View service logs:

```
sudo journalctl -u automation-controller-task -f
sudo podman logs automation-controller-web
```

Common Containerized Installation Issues

Receptor port blocked (execution nodes not connecting)

Ensure port 27199 is open in the firewall on all nodes:

```
sudo firewall-cmd --permanent --zone=public --add-port=27199/tcp
sudo firewall-cmd --reload
```

Verify Receptor is listening:

```
ss -ntlp | grep 27199
```

SELinux denials

Check for SELinux audit denials:

```
sudo ausearch -m avc -ts recent
```

If denials are present, generate and apply a local policy, or set the boolean that allows container networking if applicable.

Image pull failures

Ensure the host is registered to Red Hat and has access to `registry.redhat.io`:

```
sudo podman login registry.redhat.io
```

PostgreSQL connection failures

Verify the database container is running and the password in the inventory matches what was configured:

```
sudo podman exec -it automation-postgresql psql -U postgres -c "\l"
```

Congratulations! You have completed the troubleshooting guide for containerized AAP installation issues.

Summary

In this chapter we learned the following about deploying Automation Mesh on a containerized AAP 2.7 installation:

- In a containerized deployment, the Control Plane uses **Hybrid nodes** (default) and **Control nodes** running as `podman` containers on RHEL hosts — unlike OCP deployments which have no

hybrid or control nodes.

- Mesh topology is defined in the **installer inventory file** using `receptor_type` (`execution` or `hop`) and `receptor_peers` (a comma-separated hostname list) — not through the AAP UI.
- **Execution nodes** (`receptor_type=execution`) run jobs using `ansible-runner` with `podman` isolation; **Hop nodes** (`receptor_type=hop`) route Receptor traffic and cannot execute automation.
- The fork capacity model applies: 1 CPU = 4 forks and 100 MB RAM = 1 fork, with `capacity_adjustment` controlling the balance.
- Sizing is expressed in terms of host VM specifications (CPU, RAM, disk IOPS) rather than pod resource requests.
- EDA activation limits and memory caps are configured via `/etc/ansible-automation-platform/eda/settings.yaml`, using `MAX_RUNNING_ACTIVATIONS` and `PODMAN_MEM_LIMIT`.
- Troubleshooting uses the `-vvv` verbosity flag on the installer, `podman ps`, `systemctl status`, and `journalctl` — not `oc` CLI commands.
- Common issues include blocked port 27199, SELinux denials, image pull failures from `registry.redhat.io`, and insufficient RAM/disk preflight failures.
- **PostgreSQL tuning** is critical at scale: set `shared_buffers` to 25% of RAM, size `max_connections` to cover all AAP component connection pools, and set `checkpoint_completion_target=0.9` to reduce write I/O spikes.
- **EDA horizontal scaling**: separate API and worker roles using `eda_type=api` and `eda_type=worker` in the `[automationeda]` inventory group to scale activation execution independently of API handling.
- Use key performance indicators (high API latency, 502 gateway errors, PostgreSQL `too many clients`, activation queue buildup) to determine whether to scale vertically, add nodes, or tune configuration.

Appendix

AAP 2.7 Documentation

- Ansible Automation Platform 2.7 Documentation
- Administer Ansible Automation Platform 2.7

Automation Mesh

- Automation Mesh for Operator (OCP) Environments
- Automation Mesh for Containerized and VM Environments

Installation

- Containerized AAP Installation Guide
- Deploying AAP on OpenShift Container Platform

Sizing and Capacity Planning

- Red Hat Ansible Automation Platform Sizing Guide

Performance Optimization

- Optimize Ansible Automation Platform 2.7